UCC 152-2

Introduction to security in DevOps

Workshop 1: Secure Design and Implementation of the DigiBank Application

Cloud Computing Professional License

University of the Mountains

By **Engineer Willy Damtchou**

July 2026

Part 1: Context, objectives, target architecture and technical choices

1.1 General context of the workshop

As part of the UCC152-2 — Introduction to Security in DevOps course, the practical exercises are based on the DigiBank case study, which represents the core banking system of a new, fictional digital bank also named DigiBank. This project plays a central role in the course's pedagogical progression, as it provides a realistic framework for applying concepts of secure development, validation, testing, static analysis, dynamic analysis, artifact hardening, and delivery chain automation.

The choice of a banking system is not insignificant. An application of this type concentrates several requirements characteristic of modern systems: identity management, protection of personal and financial data, API exposure, maintenance of transactional integrity, traceability of operations, access control, availability requirements, and the use of industrialized deployment artifacts. The DigiBank case therefore allows us to work within a sufficiently rich scope to illustrate the logic of DevSecOps without falling into an artificially simplified scenario.

In this first workshop, the objective is not yet to systematically analyze vulnerabilities as will be the case in the SAST and DAST workshops. The primary goal is to build a first functional, clean, modular, and testable version of the application, thus establishing a coherent technical foundation for the subsequent security work. This initial step must already incorporate certain security best practices, including input validation, clear code organization, separation of responsibilities, the absence of hard-coded secrets, controlled error handling, and the implementation of automated tests.

Thus, workshop 1 plays a dual role. On the one hand, it has students produce a concrete banking application using Java and Spring Boot. On the other hand, it establishes the structural foundation that will serve as the basis for analysis in workshops 2, 3, and 4, which will focus respectively on SAST, DAST, and then container and dependency security.

1.2 Pedagogical positioning of the workshop

This workshop serves as the practical entry point to the DevSecOps track of the course. In a traditional development approach, students might be tempted to quickly code

an application, focusing first on making the functionalities visible. Such a logic often produces a functional system that is difficult to test, maintain, and secure. This course, however, advocates a different approach: building a clean, organized, and testable foundation from the outset, because a poorly structured application quickly becomes very costly to fix when security requirements need to be integrated.

The workshop's philosophy is therefore clearly **Shift Left** . Even though SAST and DAST analysis will only be covered in subsequent workshops, students should already be developing the habit of designing projects in which security can be seamlessly integrated. This means that architectural quality, clear responsibilities, layer separation, centralized exception handling, the use of declarative validations, and the presence of tests are not secondary refinements: they are prerequisites for an effective DevSecOps pipeline.

From a pedagogical standpoint, this first workshop should also reassure the student. Before looking for vulnerabilities, one must first have a working system. Before applying analysis tools, one must have code that one understands. Before running dependency scans or dynamic tests, one must have a stable project that is locally executable, well-documented, and sufficiently modular so that each part can be examined systematically.

1.3 Purpose of the solution to be produced

The expected solution in Workshop 1 is a first executable version of DigiBank in the form of a modular, multi-module monolith based on Spring Boot. This choice is important. The goal is not yet to produce a microservices architecture, nor to distribute the system across several independent services. The aim is to maintain a unified deployment, in order to reduce operational complexity for students, while enforcing a strict modular organization within the project.

Such a solution offers several pedagogical advantages. First, it allows students to work across multiple business domains without immediately having to deal with the challenges of distribution, service discovery, or inter-service coordination. Second, it makes the system's internal structure more visible, as each module must have its own layers, business components, and tests. Finally, it provides an ideal foundation for future workshops, as it can be progressively subjected to security analyses without requiring students to shift their architectural paradigm too early.

The version produced must be sufficient to perform the following operations:

- Client creation and consultation
- Creating and viewing accounts
- Making simple transfers
- Consulting a basic history of transactions
- Exposure of documented REST APIs
- Providing a simple home view showcasing DigiBank and giving access to API documentation.

The goal is not to replicate the full functional richness of an industrial core banking system. The goal is to build a sufficiently realistic educational core to reveal the real challenges of secure application development: data integrity, input validation, layered organization, transactional consistency, interface documentation, testability, and readiness for automation.

1.4 General objective of the workshop

The overall objective of the workshop is to design, implement, test and prepare for deployment a DigiBank application in a modular monolithic Spring Boot architecture, integrating from the first version the foundations necessary for a coherent DevSecOps approach.

In other words, at the end of this first stage, the student should not simply have “a project that gets started.” They should have a structured, coherent, executable, documented, testable project, ready to be enhanced or audited. This distinction is essential, because the workshop's aim is not to produce a disposable prototype, but a solid working basis upon which subsequent workshops can build without complete reconstruction.

1.5 Specific Objectives

At the end of this workshop, the student should be able to:

A1.1: Explain the role of the DigiBank project in the progression of the course and justify the choice of a modular monolithic architecture for a first educational implementation.

A1.2: Create a parent Maven Spring Boot project organized into several business modules, with a clear separation of responsibilities.

A1.3: Set up the fundamental layers of each module, including entities, repositories, services, controllers, DTOs, validations and exceptions.

A1.4: Expose consistent REST APIs for DigiBank's core operations.

A1.5: Produce at least one index view presenting the application and providing access to the Swagger/OpenAPI documentation.

A1.6: Write unit tests with JUnit and integration scenarios with Cucumber.

A1.7: Prepare the application for execution in a Docker container and for automation in a GitHub Actions pipeline.

A1.8: Deliver a sound, stable and usable base for future SAST, DAST and security workshops for dependencies and containers.

1.6 Retained functional scope

The functional scope of this first version must remain intentionally limited, so that the workshop remains feasible within a pedagogical framework while being comprehensive enough to prepare for subsequent sessions. The DigiBank application implemented in Workshop 1 will therefore cover a reduced but significant core functionality. The selected functions are as follows:

- Customer management
- Account management
- Executing simple transfers between accounts
- Viewing transaction history
- Viewing a system homepage
- Access to automatically generated API documentation.

This scope is a relevant choice for several reasons. It introduces sensitive data, relationships between business objects, validation and transactional consistency rules, and API exposure that will later be used for dynamic testing. It also provides a concrete basis for the project's modular structure, as each domain can be associated with a distinct module or subdomain.

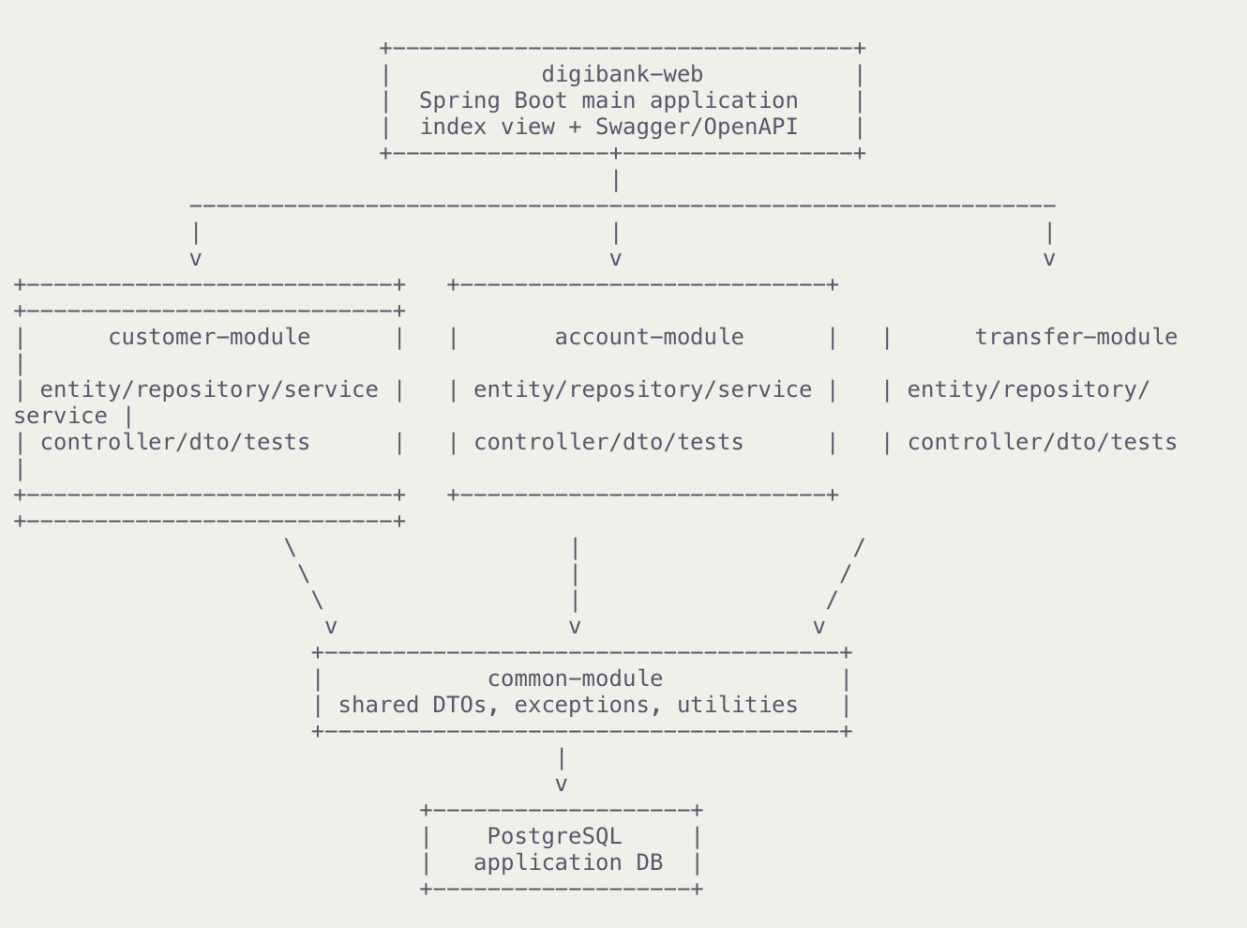
1.7 Selected target architecture

The architecture chosen for this workshop is a modular, multi-module monolith. It consists of a single Spring Boot application, packaged and executed as a whole, but structured into several Maven modules to enforce a clear separation between technical and business responsibilities. This architecture comprises a parent build module, a web bootstrap module, several business modules, and a shared common module. A typical organization would be as follows:

- **digibank-parent:** Maven parent
- **digibank-web:** application entry point, Spring Boot configuration, index view, module aggregation, OpenAPI documentation
- **common-module:** shared classes, common exceptions, generic API responses, utilities
- **customer-module:** customer management
- **account-module:** account management
- **transfer-module:** management of transfers and transaction log.

Each business module must include its own main layers: model, repository, service, controller, DTO, validation, unit tests, and integration tests where relevant. This constraint is not merely decorative. It forces students to think in terms of complete business responsibility, rather than simply piling all the code into a single, undifferentiated application package.

The logical overview can be described as follows:



This architectural form allows for the simplicity of a single deployment while intellectually preparing students for more advanced deconstructions. It also demonstrates that a monolith is not necessarily synonymous with disorder, provided it is designed with discipline.

1.8 Why not start directly with microservices?

It might seem tempting, in a course covering cloud computing, DevOps, and security, to start immediately with a distributed architecture. However, this choice would introduce additional complexity too early, hindering the learning objectives of Workshop 1. A microservices architecture raises new concerns: interservice communication, routing, distributed observability, management of multiple deployments, orchestration, fault tolerance, and consistency between services.

However, the goal of this first workshop is not to address this complexity. It is to obtain a stable, readable, and testable functional foundation. The modular monolith serves as the ideal pedagogical compromise here: it allows for the establishment of business boundaries, the development of application layers, the documentation of APIs, the writing of tests, and the preparation of security measures, without dispersing students into premature distributed infrastructure problems.

This choice is therefore not a technical step backward, but a reasoned progression. It allows us to build before distributing, to organize before industrializing, and to understand before multiplying points of failure.

1.9 Spring Boot Selection

The choice of Spring Boot for workshop 1 is based on several pedagogical and technical criteria. First, Spring Boot allows for the rapid launch of an executable project while minimizing configuration friction. Second, it provides a rich foundation for the classic layers of a modern backend application: dependency injection, REST API, validation, persistence, testing, documentation, and packaging.

For a DevSecOps workshop, Spring Boot also offers a significant methodological advantage: it integrates easily with quality assurance, testing, containerization, and automation tools. Students can therefore work in a coherent environment where the same project serves for development, JUnit and Cucumber testing, OpenAPI documentation, Docker execution, and GitHub Actions pipeline orchestration.

Finally, this choice ensures a natural continuity with the rest of the process. A well-structured Spring Boot application can easily undergo SAST scans, DAST campaigns, dependency checks, and judicious containerization. The framework is therefore not only chosen for speed, but also to serve as a credible foundation for a complete DevSecOps pipeline.

1.10 Choice of main technologies

The technologies selected for workshop 1 are as follows:

- **Java 17** as the base version of the language, to benefit from a modern, stable environment and broadly compatible with the Spring Boot ecosystem.

- **Maven** for managing builds, dependencies, and multi-module organization.
- **Spring Boot** for application booting and backend layer organization.
- **Spring Web** for REST controllers.
- **Spring Data JPA** for data access.
- **Spring Validation** for declarative validations on incoming data.
- **PostgreSQL** for the application's relational database.
- **Springdoc OpenAPI / Swagger UI** for interactive API documentation.
- **JUnit 5** and **Mockito** for unit testing.
- **Cucumber** for behavior-oriented integration testing.
- **Docker** for containerizing the application.
- **GitHub Actions** for automating compilation, testing, and some basic checks.

This choice of tools is consistent with the course's trajectory. It allows the building blocks that will be reused in subsequent workshops to be introduced from the very first workshop, without having to completely redefine the environment at each stage.

1.11 PostgreSQL Selection

The chosen database is PostgreSQL. This choice is justified both academically and practically. PostgreSQL is a robust, mature relational system widely used in modern architectures. It allows working with linked entities, integrity constraints, transactions, and a schema structure adapted to a simplified banking scenario.

In DigiBank, even though the functional scope remains largely educational, significant business relationships already exist: a customer can have multiple accounts, an account can be involved in transactions, a transfer handles several references and requires a minimum level of data consistency. A relational database management system (RDBMS) like PostgreSQL is therefore particularly well-suited for this initial implementation.

The use of PostgreSQL is also relevant to the rest of the course. Database dependencies, configuration files, environment variables, and containerization artifacts related to PostgreSQL will provide concrete material for subsequent workshops on dependency, configuration, and container security.

1.12 Selection of tests to be integrated from workshop 1

From this initial stage, the project must incorporate two levels of testing: unit tests with JUnit and integration or behavioral tests with Cucumber. This choice is essential, as it embodies the idea that quality and security cannot be added as an afterthought.

Unit tests will be used to verify fundamental business rules within the services, such as the correct creation of a customer, the opening of an account, or the rejection of a transfer due to insufficient funds. Cucumber scenarios will allow for the description of more general functional behaviors, such as creating a customer and then opening an associated account, or executing a transfer between two existing accounts.

Introducing these two forms of testing from workshop 1 onwards offers several advantages. It improves the reliability of the application platform, prepares for future integration into GitHub Actions, and provides students with observable evidence of the system's proper functioning before entering the security analysis workshops.

1.13 Choosing an index view and Swagger

Even though the core of DigiBank in this workshop is a backend application, it's useful to include a simple home screen. This screen isn't intended to be a full frontend, but rather to serve an educational purpose: introducing the project, briefly outlining its features, and providing a direct link to the API documentation.

This decision has two advantages. Firstly, it provides a clear entry point for learners or teachers running the project for the first time. Secondly, it embodies a useful practice in educational projects: offering a simple yet informative start page, rather than a backend with no visible markers.

The integration of **Swagger/OpenAPI** is equally important. It allows for the automatic documentation of endpoints exposed by DigiBank, the testing of routes from a graphical interface, and the preparation of future dynamic verification scenarios. A documented API becomes easier to understand, simpler to evaluate, and more practical to submit to systematic testing.

1.14 Docker and GitHub Actions Placement in Workshop 1

Although container and dependency security will be studied in depth in workshop 4, it is pedagogically useful to introduce a first version of the Dockerfile, a simple docker-compose.yml and a minimal GitHub Actions pipeline in workshop 1.

This decision is based on a practical DevSecOps approach. A modern project isn't limited to its Java classes. It also includes the artifacts that enable its construction, execution, testing, and deployment. By providing students with an initial automated execution pipeline now, we prevent them from perceiving Docker and GitHub Actions as peripheral additions. Instead, they see them as normal components of the software environment.

In workshop 1, Docker and GitHub Actions will not yet be fully utilized. Their role will be to ensure initial industrialization: launching the PostgreSQL database, running the application in a container if necessary, compiling the project, running tests, and verifying that the project is at least reproducible in a simple pipeline. More advanced hardening, scanning, and security controls will be explored in greater depth later.

1.15 Advantages of the chosen architecture

The architecture chosen for workshop 1 offers several educational and technical advantages:

- It remains simple enough to be understood and assembled in a guided workshop.
- It already imposes a real internal structuring of the code.
- It allows for separate testing of business domains.
- It naturally prepares the ground for future SAST and DAST workshops.
- It integrates easily with Docker and GitHub Actions.
- It provides a clean foundation for a possible future evolution towards a more distributed architecture.

In other words, this architecture seeks a balance between simplicity of execution and rigor of design. It does not try to impress with sophistication, but to lay solid, legible and usable foundations.

1.16 Acknowledged limitations of this first version

Like any first educational implementation, the solution built in workshop 1 has acknowledged limitations. It does not yet implement full authentication, granular role-based authorization, advanced encryption, deep observability, distributed orchestration, or complex high-availability mechanisms.

These limitations are not oversights. They are intentional. The course was specifically structured into several workshops so that each dimension would be addressed at the appropriate time. It would be counterproductive to try to introduce everything in the first project, risking making it too cumbersome, too opaque, or too fragile for a gradual learning process.

However, this first version must be clean enough to support these enhancements later. That's why the emphasis is being placed right now on structure, consistency, validation, testing, documentation, and reproducibility.

1.17 Expected result at the end of part 1

At the end of this first part, the student should have understood what he is going to build, why this architecture was chosen, what technologies will be used, and how this first base fits into the overall logic of the course.

He must be able to explain:

- Why DigiBank is a good educational tool for DevSecOps;
- Why we start with a modular monolith rather than microservices;
- Why Spring Boot, PostgreSQL, JUnit, Cucumber, Docker and GitHub Actions were chosen;
- Why the quality of the initial architecture influences the quality of subsequent workshops.

This understanding is essential, because in the next part, the workshop will switch to concrete execution: setting up the environment, creating the project in IntelliJ IDEA, multi-module tree structure, choosing dependencies and initial project configurations.

Part 2: Environment, software prerequisites, project creation in IntelliJ IDEA, multi-module structure, Maven commands and initial configurations

2.1 Role of this phase in the workshop

Before developing any business functionality for DigiBank, students must have a consistent, reproducible, and properly configured working environment. This requirement is particularly important in a DevSecOps workshop, as a significant portion of the difficulties stem not only from the code itself, but also from incomplete installations, incompatible versions, misconfigured paths, or a poorly structured project from the outset.

This second part therefore has a foundational function. It doesn't simply consist of "installing tools," but rather of setting up the entire technical framework that will subsequently allow for the development, testing, documentation, packaging, and automation of DigiBank. The time invested here is strategic: a project properly created in

IntelliJ IDEA, correctly organized into Maven modules, and connected to a PostgreSQL database will avoid many roadblocks in the following phases.

Students must understand that a serious DevSecOps project always begins with a solid technical foundation. A stable environment promotes reproducibility, collaboration, and future automation, while a cobbled-together environment introduces technical debt, inconsistencies, and errors that are difficult to diagnose later.

2.2 Software prerequisites to install

To successfully complete workshop 1, the student will need to install or check the following components:

- Java JDK 17
- Apache Maven 3.9 or compatible version
- IntelliJ IDEA Community or Ultimate
- Git
- PostgreSQL
- Docker Desktop or Docker Engine with Docker Compose
- A modern web browser
- Possibly Postman for preliminary manual testing.

These tools do not all have the same function, but they form a coherent whole. The JDK allows the application to be compiled and run, Maven structures the project and manages dependencies, IntelliJ IDEA serves as the main development workstation, PostgreSQL hosts the data, Docker prepares for containerization, Git manages version control, and GitHub Actions will later be used to automate the build and testing process.

2.3 Recommended versions

To minimize compatibility issues, it is recommended to use the following versions:

Tool	Recommended version	Role

Java	JDK 17	Compilation and execution
Maven	3.9.x	Multi-module build
IntelliJ IDEA	2024.x or later	Development
Spring Boot	Java 17 compatible 3.x	Application framework
PostgreSQL	15 or 16	Relational database
Docker	recent stable version	Containerization
Git	recent stable version	Source code management

This standardization of versions greatly simplifies the practical work in class. It prevents some students from working with software combinations that are too old or too heterogeneous, which would harm the reproducibility of the project.

2.4 Verification of installations

Before creating DigiBank, each student must verify that their environment is functioning correctly. The following commands can be executed in a terminal:

```
Java -version

mvn -version

git --version
```

```
docker --version

docker compound version

psql --version
```

If all commands correctly return an installed version, the basic environment is ready. If any of them fail, the student must correct the problem before proceeding; otherwise, the following steps may fail misleadingly.

On Windows, it may be necessary to check the JAVA_HOME variable and the presence of executables in the PATH. On Linux or macOS, you must ensure that the current shell can see the installed tools.

.

2.5 Recommended development workstation configuration

To streamline the work process, it is recommended to prepare a clean workspace, for example a main directory dedicated to course projects:

```
mkdir -p ~/projects/ucc152

CD ~/projects/ucc152
```

In this directory, the DigiBank project will be created in a single folder. This organization prevents file scattering and facilitates opening the project in IntelliJ IDEA, Git management, Docker scripts, and Maven commands.

It is also recommended to enable the following in IntelliJ IDEA:

- Automatic import of Maven projects
- Automatic downloading of source code and documentation
- UTF-8 encoding
- Displaying hidden folders and the complete project structure
- Integrated Git support.

2.6 Creating the PostgreSQL database

Before connecting the application to the persistent storage, the database that will host DigiBank must be created. A simple configuration can be used for educational purposes:

- Database name: **digibankdb**
- User: **digibank**
- Password: **digibank123**

From psql, the student can execute:

```
CREATE DATABASE digibankdb ;

CREATE USER digibank WITH ENCRYPTED PASSWORD 'digibank123' ;

GRANT ALL PRIVILEGES ON DATABASE digibankdb TO digibank;
```

If PostgreSQL doesn't immediately allow certain operations from another client, you'll also need to check the settings in pg_hba.conf and the local authentication method. For the purposes of this lab, this configuration is intentionally kept simple, but it will need to be more robust later in the workshops dedicated to secret security, dependencies, and containerization.

2.7 Quickly launching a database with Docker

If the student does not wish to use a local PostgreSQL installation or if the computer environment is unstable, they can launch a PostgreSQL database via Docker. A simple command is all that is needed:

```
docker run -d \

 --name digibank-postgres \

 --restart unless stopped \

 -e POSTGRES_DB=digibankdb \

 -e POSTGRES_USER=digibank \

 -e POSTGRES_PASSWORD=digibank123 \

 -p 5432:5432 \
```

```
-v digibank-postgres-data:/var/lib/postgresql/data \

postgres:16
```

This option is very useful in an educational context, as it guarantees a reproducible and quickly resettable basic environment. It also naturally introduces Docker from the very first workshop, without yet delving into advanced image hardening practices.

To verify that the container is working:

```
docker ps
```

To consult the newspapers:

```
docker logs digibank-postgres
```

2.8 Creating the project in IntelliJ IDEA

The DigiBank project must be created rigorously. In IntelliJ IDEA, the student must follow this logic:

1. Open IntelliJ IDEA.
2. Select New Project.
3. Select Maven as the base project type.
4. Select the installed JDK 17.
5. Define the parent project information:
6. Group ID: com.m2ibank
7. ArtifactId: digibank-parent
8. Version: 1.0.0-SNAPSHOT
9. Check the option to create the project from an empty archetype if necessary.
10. Choose the working folder, for example ~/projects/ucc152/digibank-parent.
11. Finish creation.

At this stage, the idea is not to directly create a single Spring Boot application, but first a Maven parent project designed to manage several modules. This distinction is important:

the project root will not directly contain the business logic; it will serve as the orchestrator for the multi-module build.

2.9 Why start with a Maven parent

In a multi-module architecture, the Maven parent plays a structuring role. It centralizes the project version, global properties, dependency management, common plugins, and sub-module declarations. Without this parent, each module would have to repeat a significant portion of its configuration, making the project heavier, less readable, and more difficult to maintain.

DigiBank's Maven parent will therefore allow:

- To impose a common version of Java
- To fix the Spring Boot version
- To centralize test versions
- To declare the project modules
- To simplify global build commands.

This approach perfectly matches the educational objective of the workshop: to show that a serious project is structured from its very beginning.

2.10 Target Project Tree

Once the parent project is created, the student must understand the expected final structure. The target directory structure will be as follows:

```
digibank-parent/
├── pom.xml
├── common-module/
│   ├── pom.xml
│   └── src/
├── customer-module/
│   ├── pom.xml
│   └── src/
├── account-module/
│   ├── pom.xml
│   └── src/
├── transfer-module/
│   ├── pom.xml
│   └── src/
└── digibank-web/
    ├── pom.xml
    └── src/
```

This structure reflects the choice of a modular monolith. The code is separated by responsibility, but the whole is assembled and executed as a single application.

2.11 Creating the parent pom.xml file

The first crucial file to configure is the root pom.xml. A consistent base version for this workshop is as follows:

```
<project xmlns ="http://maven.apache.org/POM/4.0.0"
```

```
        xmlns: xsi ="http://www.w3.org/2001/XMLSchema-instance"
        xsi :schemaLocation ="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd

<modelVersion> 4.0.0 </modelVersion>

<groupId> com.m2ibank </groupId>
<artifactId> digibank-parent </artifactId>
<version> 1.0.0-SNAPSHOT </version>
<packaging> pom </packaging>
<name> digibank-parent </name>
<description>   DigiBank   modular   monolith   parent   project
</description>

<modules>
<module> common-module </module>
<module> customer-module </module>
<module> account-module </module>
<module> transfer-module </module>
<module> digibank-web </module>
</modules>
```

```
<properties>

<java.version> 17 </java.version>

<spring.boot.version> 3.3.2 </spring.boot.version>

<springdoc.version> 2.6.0 </springdoc.version>

<cucumber.version> 7.18.0 </cucumber.version>

<junit.version> 5.10.2 </junit.version>

<maven.compiler.plugin.version>                                    3.13.0
</maven.compiler.plugin.version>

</properties>



<dependencyManagement>

<dependencies>

<dependency>

<groupId> org.springframework.boot </groupId>

<artifactId> spring-boot-dependencies </artifactId>

<version> ${spring.boot.version} </version>

<type> pom </type>

<scope> import </scope>

</dependency>

</dependencies>

</dependencyManagement>
```

```
<build>

<pluginManagement>

<plugins>

<plugin>

<groupId> org.apache.maven.plugins </groupId>

<artifactId> maven-compiler-plugin </artifactId>

<version> ${maven.compiler.plugin.version} </version>

<configuration>

<source> ${java.version} </source>

<target> ${java.version} </target>

</configuration>

</plugin>

</plugins>

</pluginManagement>

</build>

</project>
```

2.12 Creating sub-modules in IntelliJ IDEA

In IntelliJ IDEA, the student can create each sub-module via:

1. Right-click on the parent project
2. New
3. Module
4. Maven type

5. Parent's inheritance
6. Choosing the module name.

The modules to be created are:

- **common-module**
- **customer-module**
- **account-module**
- **transfer-module**
- **digibank-web**

After creation, IntelliJ IDEA automatically or semi-automatically updates the Maven parent if Maven import is enabled. If not, you must verify that each module is correctly listed in the <modules> section of the root pom.xml file.

2.13 Creating sub-modules in IntelliJ IDEA

Each business module must declare its parent module membership. A basic template for customer-module/pom.xml can be used as a model for other modules:

```
<project xmlns ="http://maven.apache.org/POM/4.0.0"
        xmlns: xsi ="http://www.w3.org/2001/XMLSchema-instance"
        xsi :schemaLocation ="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd


<modelVersion> 4.0.0 </modelVersion>


<parent>
<groupId> com.m2ibank </groupId>
<artifactId> digibank-parent </artifactId>
<version> 1.0.0-SNAPSHOT </version>
</parent>
```

```
<artifactId> customer-module </artifactId>
<name> customer-module </name>
<packaging> jar </packaging>

<dependencies>
<dependency>
<groupId> com.m2ibank </groupId>
<artifactId> common-module </artifactId>
<version> ${project.version} </version>
</dependency>

<dependency>
<groupId> org.springframework.boot </groupId>
<artifactId> spring-boot-starter-data-jpa </artifactId>
</dependency>

<dependency>
<groupId> org.springframework.boot </groupId>
<artifactId> spring-boot-starter-validation </artifactId>
</dependency>

<dependency>
<groupId> org.springframework.boot </groupId>
```

```
<artifactId> spring-boot-starter-web </artifactId>
</dependency>

<dependency>
<groupId> org.postgresql </groupId>
<artifactId> postgresql </artifactId>
<scope> runtime </scope>
</dependency>

<dependency>
<groupId> org.springframework.boot </groupId>
<artifactId> spring-boot-starter-test </artifactId>
<scope> test </scope>
</dependency>

<dependency>
<groupId> org.mockito </groupId>
<artifactId> mockito-core </artifactId>
<scope> test </scope>
</dependency>
</dependencies>
</project>
```

The account-module and transfer-module modules will follow the same logic, with dependencies adapted to their needs.

2.13 pom.xml file of the common-module module

The common module is intended to contain the cross-cutting elements. Its pom.xml file can remain smaller:

```
<project xmlns ="http://maven.apache.org/POM/4.0.0"
        xmlns: xsi ="http://www.w3.org/2001/XMLSchema-instance"
        xsi :schemaLocation ="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd

<modelVersion> 4.0.0 </modelVersion>

<parent>
<groupId> com.m2ibank </groupId>
<artifactId> digibank-parent </artifactId>
<version> 1.0.0-SNAPSHOT </version>
</parent>

<artifactId> common-module </artifactId>
<name> common-module </name>
<packaging> jar </packaging>

<dependencies>
<dependency>
<groupId> org.springframework.boot </groupId>
```

```
<artifactId> spring-boot-starter-validation </artifactId>

</dependency>

</dependencies>

</project>
```

This module will later contain common exceptions, some standardized API responses, and possibly shared objects.

2.15 pom.xml of the digibank-web module

The digibank-web module will be the application's entry point. It will depend on the other modules:

```
<project xmlns ="http://maven.apache.org/POM/4.0.0"

        xmlns: xsi ="http://www.w3.org/2001/XMLSchema-instance"

        xsi :schemaLocation ="http://maven.apache.org/POM/4.0.0

https://maven.apache.org/xsd/maven-4.0.0.xsd


<modelVersion> 4.0.0 </modelVersion>


<parent>

<groupId> com.m2ibank </groupId>

<artifactId> digibank-parent </artifactId>

<version> 1.0.0-SNAPSHOT </version>

</parent>


<artifactId> digibank-web </artifactId>

<name> digibank-web </name>
```

```
<packaging> jar </packaging>


<dependencies>
<dependency>
<groupId> com.m2ibank </groupId>
<artifactId> common-module </artifactId>
<version> ${project.version} </version>
</dependency>
<dependency>
<groupId> com.m2ibank </groupId>
<artifactId> customer-module </artifactId>
<version> ${project.version} </version>
</dependency>
<dependency>
<groupId> com.m2ibank </groupId>
<artifactId> account-module </artifactId>
<version> ${project.version} </version>
</dependency>
<dependency>
<groupId> com.m2ibank </groupId>
<artifactId> transfer-module </artifactId>
<version> ${project.version} </version>
</dependency>
```

```
<dependency>
<groupId> org.springframework.boot </groupId>
<artifactId> spring-boot-starter-thymeleaf </artifactId>
</dependency>

<dependency>
<groupId> org.springframework.boot </groupId>
<artifactId> spring-boot-starter-web </artifactId>
</dependency>

<dependency>
<groupId> org.springframework.boot </groupId>
<artifactId> spring-boot-starter-data-jpa </artifactId>
</dependency>

<dependency>
<groupId> org.springframework.boot </groupId>
<artifactId> spring-boot-starter-validation </artifactId>
</dependency>

<dependency>
<groupId> org.springdoc </groupId>
<artifactId> springdoc-openapi-starter-webmvc-ui </artifactId>
<version> ${springdoc.version} </version>
```

```
</dependency>


<dependency>
<groupId> org.postgresql </groupId>
<artifactId> postgresql </artifactId>
<scope> runtime </scope>
</dependency>


<dependency>
<groupId> org.springframework.boot </groupId>
<artifactId> spring-boot-starter-test </artifactId>
<scope> test </scope>
</dependency>
</dependencies>


<build>
<plugins>
<plugin>
<groupId> org.springframework.boot </groupId>
<artifactId> spring-boot-maven-plugin </artifactId>
</plugin>
</plugins>
</build>
</project>
```

This module is essential, as it is the one that will assemble the business dependencies and expose the complete application to the runtime.

2.16 Creating src trees

In each module, you need to create the standard Maven directory structure:

```
src/
├── main/
│   ├── java/
│   └── resources/
└── test/
    ├── java/
    └── resources/
```

For business modules, the recommended organization under src/main/java is as follows:

```
com/m2ibank/customer/
├── controller/
├── dto/
├── entity/
├── repository/
├── service/
├── exception/
└── mapper/
```

The same model will be used for account-module and transfer-module. This ensures visual and structural consistency throughout the project.

2.17 First Maven build order

Once the parent structure and modules are created, the student can test the global build:

```
CD digibank-parent

mvn clean install
```

Initially, this build may fail if certain classes or configurations are still missing. But after the minimum POMs are correctly set up, the command should at least resolve dependencies and traverse the build tree.

When the project is more advanced, this command will become the standard consistency check for the entire project.

2.18 Added .gitignore

From the outset, the project must be properly versioned. A .gitignore file must be added to the root directory:

```
target/

!.mvn/wrapper/maven-wrapper.jar

!**/src/main/**/target/

!**/src/test/**/target/

.kotlin


### IntelliJ IDEA ###

.idea/modules.xml

.idea/jarRepositories.xml

.idea/compiler.xml

.idea/libraries/
```

```
*.iws

*.iml

*.ipr


### Eclipse ###

.apt_generated

.classpath

.factorypath

.project

.settings

.springBeans

.sts4-cache


### NetBeans ###

/nbproject/private/

/nbbuild/

/dist/

/nbdist/

/.nb-gradle/

build/

!**/src/main/**/build/

!**/src/test/**/build/


### VS Code ###
```

```
.vscode/


### Mac OS ###
.DS_Store


### Logs ###
logs/
*.log


### Env files ###
.env
```

This file prevents the Git repository from being cluttered with temporary files, build artifacts, or local configurations specific to a machine. In a DevSecOps context, this practice is important because a poorly maintained repository quickly becomes a source of confusion, noise, and sometimes even leaks of sensitive information.

2.19 Git initialization of the project

Once the project is created, the student can initialize Git:

```
git init

git add .

git commit -m "Initialize DigiBank multi-module project structure"
```

This step should be done early, as it helps to keep a clean history and easily return to a stable state if a bad configuration occurs later.

2.20 Initial application configuration.yml

The digibank-web module will contain the application's core configuration. A first version can be placed in digibank-web/src/main/resources/application.yml:

```
spring :
 application :
   name : digibank
 datasource :
   url : jdbc:postgresql://localhost:5432/digibankdb
   username : digibank
   password : digibank123
   driver-class-name : org.postgresql.Driver
 jpa :
   hibernate :
     ddl-auto : update
   show-sql : true
   properties :
     hibernate :
       SQL format : true
 thymeleaf :
   cache : false


server :
```

```
port : 8080


springdoc :
 api-docs :
   path : /api-docs
 swagger-ui :
   path : /swagger-ui.html
```

This configuration is simple but sufficient to get started. It will be refined later, particularly to better separate profiles, better manage secrets, and prepare for containerization.

2.21 Initial application configuration.yml

The YAML format is particularly useful for Spring Boot applications because it allows parameters to be structured neatly into hierarchical blocks. In an educational project like DigiBank, it also improves the readability of settings for students.

It's important to remember that a configuration file should not be treated as a secure vault. The credentials used here are for the sake of simplicity in this exercise, but they do not represent acceptable production practices. This limitation will be addressed in future workshops on security.

2.22 Creating the Spring Boot main class

In digibank-web/src/main/java/com/m2ibank/web/, the student will later create the main class, but it is useful to plan its location now. The class can be called DigiBankApplication.

The main class package must be high enough in the directory tree to allow scanning of Spring components in business modules. This decision is important because an incorrect placement of the main package often leads to errors in detecting beans, controllers, or repositories.

2.23 Verification of Maven import in IntelliJ

After all POM modifications, you must ask IntelliJ IDEA to reload the Maven project. In the Maven view:

1. Click on Reload All Maven Projects
2. Verify that all modules appear
3. Verify that the Spring Boot dependencies are resolved correctly.
4. Check that no POM is reported as invalid.

This step is crucial. A multi-module project that is incorrectly reloaded into the IDE can give the illusion that the code or imports are wrong, when the real problem simply comes from an incomplete Maven synchronization.

2.24 Useful Maven commands in this phase

The following commands must be known and used by students:

```
mvn clean

mvn compile

mvn test

mvn clean install

mvn dependency:tree
```

Their role is as follows:

- **clean** removes artifacts from previous builds
- **compile** checks the compilation of the main code
- **test** launches tests
- **clean install** rebuilds and installs the modules locally
- **dependency:tree** helps to visualize transitive dependencies

The habit of running these commands regularly is important in a DevSecOps approach, as it reinforces the reflex of continuous verification and reproducible builds.

2.25 Expected work structure at the end of this part

At the end of this second part, the student should have:

- From a complete and verified software environment
- From a local or containerized PostgreSQL database
- From a multi-module Maven parent project created in IntelliJ IDEA
- Of the five modules declared and linked to the parent
- The first correctly configured pom.xml files
- From an initial application.yml file
- From an initialized Git repository
- From a structure ready to receive the business code.

In other words, at this stage, the groundwork must be completely laid. The student is not yet building the entire application, but has established a sufficiently clean and stable environment to begin development without structural improvisation.

Part 3: Detailed Implementation of the Source Code

3.1 General Implementation Logic

In this workshop, DigiBank remains a modular monolith: the application is deployed as a single system, but the code is organized vertically by business domains. Each module therefore carries its own internal layers, which allows students to see that a well-designed monolith can already be structured, testable, readable, and scalable.

The first three functional areas selected for this initial version are:

- Customer management
- Account management
- Managing wire transfers.

The common-module module will handle the cross-cutting elements, while digibank-web will serve as the assembly module and entry point for the application. An index view will also be added later to showcase DigiBank and provide access to the Swagger API documentation.

3.2 Overview of modules and responsibilities

The chosen functional distribution is as follows:

Module	Main responsibility
`common-module`	Common exceptions, response objects, shared utilities
`customer-module`	Client creation and consultation
`account-module`	Bank account management
`transfer-module`	Beneficiary and transfer management
`digibank-web`	Spring Boot entry point, global configuration, index view, Swagger

This structure clearly shows that each business domain has its own layers, instead of a single horizontal technical architecture. This is precisely what you wanted to implement: vertical modules, each with its own complete logic.

3.3 Implementation of the common-module

The common module must be created first, as other modules will rely on it for exceptions and standardized API responses.

3.3.1 ApiResponse Class

Create the file:

common-module/src/main/java/com/m2ibank/common/api/ApiResponse.java

```java
package com.m2ibank.common.api;


import java.time.LocalDateTime;


public class ApiResponse< T > {


   private boolean success ;

   private String message ;

   private T data ;

   private LocalDateTime timestamp ;


   public ApiResponse () {

       this . timestamp = LocalDateTime. now ();

}


   public ApiResponse ( boolean success, String message, T data)
{

       this.success = success ;

       this.message = message ;

       this.data = data ;

       this . timestamp = LocalDateTime. now ();

}
```

```
   public static < T > ApiResponse< T > success (String message,
T data) {
       return new ApiResponse<>( true , message, data);
}

   public static < T > ApiResponse< T > failure (String message,
T data) {
       return new ApiResponse<>( false , message, data);
}

   public boolean isSuccess () {
       return success ;
}

   public void setSuccess ( boolean success) {
       this.success = success ;
}

   public String getMessage () {
       return message ;
}

   public void setMessage (String message) {
       this.message = message ;
}
```

```
    public T getData () {
        return data ;
}

    public void setData ( T data) {
        this.data = data ;
}

    public LocalDateTime getTimestamp () {
        return timestamp ;
}

    public void setTimestamp (LocalDateTime timestamp) {
        this.timestamp = timestamp ;
}
}
```

This class normalizes REST responses. It helps avoid inconsistent responses from one controller to another and improves API readability.

3.3.2 ResourceNotFoundException

Create the file:

common-module/src/main/java/com/m2ibank/common/exception/ResourceNotFoundException.java

```
package com.m2ibank.common.exception;


public  class  ResourceNotFoundException  extends  RuntimeException
{


   public ResourceNotFoundException (String message) {

        great (message);

}

}
```

3.3.3 BusinessException

Create the file:

common-module/src/main/java/com/m2ibank/common/exception/BusinessException.java

```
package com.m2ibank.common.exception;


public class BusinessException extends RuntimeException {


   public BusinessException (String message) {

        great (message);

}

}
```

These exceptions will be used to distinguish resource not found errors from business errors, for example an insufficient balance or an unauthorized transaction.

3.4 Implementation of the customer-module

The customer module allows for the management of basic information relating to DigiBank customers. It constitutes a logical entry point, as accounts and transfers necessarily rely on the existence of customers.

3.4.1 Customer Entity

Create the file:

customer-module/src/main/java/com/m2ibank/customer/entity/Customer.java

```
package com.m2ibank.customer.entity;


import jakarta.persistence.*;
import java.time.LocalDateTime;


@Entity
@Table (name = "customers" )
public class Customer {


   @Id
@GeneratedValue ( strategy = GenerationType.IDENTITY )
   private Long id ;


   @Column (nullable = false , length = 120 )
   private String fullName ;
```

```
    @Column (nullable = false , unique = true , length = 150 )
    private String email ;

    @Column (nullable = false , unique = true , length = 30 )
    private String phoneNumber ;

    @Column (nullable = false , length = 30 )
    private String nationalId ;

    @Column (nullable = false )
    private LocalDateTime createdAt ;

    public Customer () {
        this . createdAt = LocalDateTime. now ();
}

    public Customer (String fullName, String email, String
phoneNumber, String nationalId) {
        this.fullName = fullName ;
        this.email = email ;
        this . phoneNumber = phoneNumber;
        this . nationalId = nationalId;
        this . createdAt = LocalDateTime. now ();
}
```

```
    public Long getId () {
        return id ;
}

    public String getFullName () {
        return fullName ;
}

    public void setFullName (String fullName) {
        this.fullName = fullName ;
}

    public String getEmail () {
        return email ;
}

    public void setEmail (String email) {
        this.email = email ;
}

    public String getPhoneNumber () {
        return phoneNumber ;
}
```

```
    public void setPhoneNumber (String phoneNumber) {
        this . phoneNumber = phoneNumber;
}

    public String getNationalId () {
        return nationalId ;
}

    public void setNationalId (String nationalId) {
        this . nationalId = nationalId;
}

    public LocalDateTime getCreatedAt () {
        return createdAt ;
}
}
```

3.4.2 DTO CustomerRequest

Create the file:

customer-module/src/main/java/com/m2ibank/customer/dto/CustomerRequest.java

```
package com.m2ibank.customer.dto;

import jakarta.validation.constraints. Email ;
```

```
import jakarta.validation.constraints. NotBlank ;
import jakarta.validation.constraints. Size ;


public class CustomerRequest {


   @NotBlank (message = "Full name is required" )
   @Size (max = 120 )
   private String fullName ;


   @NotBlank (message = "Email is required" )
   @Email (message = "Email format is invalid" )
   private String email ;


   @NotBlank (message = "Phone number is required" )
   @Size (max = 30 )
   private String phoneNumber ;


   @NotBlank (message = "National ID is required" )
   @Size (max = 30 )
   private String nationalId ;


   public String getFullName () {
       return fullName ;
}
```

```
    public void setFullName (String fullName) {
        this.fullName = fullName ;
}

    public String getEmail () {
        return email ;
}

    public void setEmail (String email) {
        this.email = email ;
}

    public String getPhoneNumber () {
        return phoneNumber ;
}

    public void setPhoneNumber (String phoneNumber) {
        this . phoneNumber = phoneNumber;
}

    public String getNationalId () {
        return nationalId ;
}
```

```
    public void setNationalId (String nationalId) {
        this . nationalId = nationalId;
}
}
```

3.4.3 DTO CustomerResponse

Create the file:

customer-module/src/main/java/com/m2ibank/customer/dto/CustomerResponse.java

```
package com.m2ibank.customer.dto;

import java.time.LocalDateTime;

public class CustomerResponse {

    private Long id ;
    private String fullName ;
    private String email ;
    private String phoneNumber ;
    private String nationalId ;
    private LocalDateTime createdAt ;

    public CustomerResponse () {
}
```

```
    public CustomerResponse (Long id, String fullName, String
email, String phoneNumber, String nationalId, LocalDateTime
createdAt) {
        this.id = id ;
        this.fullName = fullName ;
        this.email = email ;
        this . phoneNumber = phoneNumber;
        this . nationalId = nationalId;
        this.createdAt = createdAt ;
}

   public Long getId () {
        return id ;
}

   public String getFullName () {
        return fullName ;
}

   public String getEmail () {
        return email ;
}

   public String getPhoneNumber () {
```

```
        return phoneNumber ;
}


    public String getNationalId () {
        return nationalId ;
}


    public LocalDateTime getCreatedAt () {
        return createdAt ;
}
}
```

3.4.4 Repository CustomerRepository

Create the file:

customer-module/src/main/java/com/m2ibank/customer/repository/CustomerRepository.java

```
package com.m2ibank.customer.repository;


import com.m2ibank.customer.entity.Customer;
import org.springframework.data.jpa.repository.JpaRepository;


import java.util.Optional;


public interface CustomerRepository extends
JpaRepository<Customer, Long> {
```

```
Optional<Customer> findByEmail (String email);

Optional<Customer> findByPhoneNumber (String phoneNumber);

}
```

3.4.5 Customer Service

Create the file:

customer-module/src/main/java/com/m2ibank/customer/service/CustomerService.java

```
package com.m2ibank.customer.service;


import com.m2ibank.common.exception.BusinessException;

import com.m2ibank.common.exception.ResourceNotFoundException;

import com.m2ibank.customer.dto.CustomerRequest;

import com.m2ibank.customer.dto.CustomerResponse;

import com.m2ibank.customer.entity.Customer;

import com.m2ibank.customer.repository.CustomerRepository;

import org.springframework.stereotype. Service ;


import java.util.List;


@Service

public class CustomerService {


   private final CustomerRepository customerRepository ;
```

```
                public      CustomerService      (CustomerRepository
customerRepository) {
        this . customerRepository = customerRepository;
}


       public   CustomerResponse   createCustomer   (CustomerRequest
request) {
                                                    customerRepository
.findByEmail(request.getEmail()).ifPresent(c -> {
               throw new BusinessException( "A customer with this
email already exists" );
});


                                                    customerRepository
.findByPhoneNumber(request.getPhoneNumber()).ifPresent(c -> {
               throw new BusinessException( "A customer with this
phone number already exists" );
});


Customer customer = new Customer(
request.getFullName(),
request.getEmail(),
request.getPhoneNumber(),
request.getNationalId()
);
```

```
Customer saved = customerRepository .save(customer);
        return mapToResponse(saved);
}

   public CustomerResponse getCustomerById (Long id) {
Customer customer = customerRepository .findById(id)
.orElseThrow(() -> new ResourceNotFoundException( "Customer not
found with id " + id ));
        return mapToResponse(customer);
}

   public List<CustomerResponse> getAllCustomers () {
        return customerRepository.findAll ()
.stream()
.map( this ::mapToResponse)
.toList();
}

   public Customer getCustomerEntityById (Long id) {
        return customerRepository .findById(id)
.orElseThrow(() -> new ResourceNotFoundException( "Customer not
found with id " + id ));
}
```

```
   private CustomerResponse mapToResponse (Customer customer) {

        return new CustomerResponse(

customer.getId(),

customer.getFullName(),

customer.getEmail(),

customer.getPhoneNumber(),

customer.getNationalId(),

customer.getCreatedAt()

);

}

}
```

3.4.6 CustomerController

Create the file:

customer-module/src/main/java/com/m2ibank/customer/controller/CustomerController.java

```
package com.m2ibank.customer.controller;


import com.m2ibank.common.api.ApiResponse;

import com.m2ibank.customer.dto.CustomerRequest;

import com.m2ibank.customer.dto.CustomerResponse;

import com.m2ibank.customer.service.CustomerService;

import jakarta.validation. Valid ;

import org.springframework.http.HttpStatus;
```

```java
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;


import java.util.List;


@RestController
@RequestMapping ( "/api/customers" )
public class CustomerController {


   private final CustomerService customerService ;


   public CustomerController (CustomerService customerService) {
       this . customerService = customerService;
}


   @PostMapping
          public    ResponseEntity<ApiResponse<CustomerResponse>>
createCustomer ( @Valid @RequestBody CustomerRequest request) {
CustomerResponse        response        =        customerService
.createCustomer(request);
       return ResponseEntity. status ( HttpStatus.CREATED )
.body(ApiResponse.success  (  "Customer  created  successfully"  ,
response));
}
```

```
   @GetMapping ( "/{id}" )

          public    ResponseEntity<ApiResponse<CustomerResponse>>
getCustomerById ( @PathVariable Long id) {

CustomerResponse         response         =         customerService
.getCustomerById(id);

             return  ResponseEntity.  ok  (ApiResponse.success  (
"Customer retrieved successfully" , response));

}


   @GetMapping

      public  ResponseEntity<ApiResponse<List<CustomerResponse>>>
getAllCustomers () {

List<CustomerResponse>       response       =       customerService
.getAllCustomers();

             return  ResponseEntity.  ok  (ApiResponse.success  (
"Customers retrieved successfully" , response));

}

}
```

3.5 Implementation of the account-module

The account module manages the bank accounts linked to the clients. It is a central module, as the transfer then relies on the source and destination accounts.

3.5.1 Enum AccountType

Create the file:

account-module/src/main/java/com/m2ibank/account/entity/AccountType.java

```
package com.m2ibank.account.entity;


public enum AccountType {
   CURRENT ,
   SAVINGS
}
```

3.5.2 Entity Account

Create the file:

account-module/src/main/java/com/m2ibank/account/entity/Account.java

```
package com.m2ibank.account.entity;


import jakarta.persistence.*;


import java.math.BigDecimal;
import java.time.LocalDateTime;


@Entity
@Table (name = "accounts" )
public class Account {


   @Id
@GeneratedValue ( strategy = GenerationType.IDENTITY )
   private Long id ;
```

```
    @Column (nullable = false , unique = true , length = 30 )
    private String accountNumber ;

    @Column (nullable = false , precision = 19 , scale = 2 )
    private BigDecimal balance ;

    @Enumerated (EnumType. STRING )
    @Column (nullable = false , length = 20 )
    private AccountType accountType ;

    @Column (nullable = false )
    private Long customerId ;

    @Column (nullable = false )
    private LocalDateTime createdAt ;

    public Account () {
        this . createdAt = LocalDateTime. now ();
}

    public Account (String accountNumber, BigDecimal balance,
AccountType accountType, Long customerId) {
        this . accountNumber = accountNumber;
```

```
        this.balance = balance ;
        this . accountType = accountType;
        this . customerId = customerId;
        this . createdAt = LocalDateTime. now ();
}

    public Long getId () {
        return id ;
}

    public String getAccountNumber () {
        return accountNumber ;
}

    public BigDecimal getBalance () {
        return balance ;
}

    public AccountType getAccountType () {
        return accountType ;
}

    public Long getCustomerId () {
        return customerId ;
```

```
}


   public LocalDateTime getCreatedAt () {
        return createdAt ;
}


   public void setBalance (BigDecimal balance) {
        this.balance = balance ;
}
}
```

3.5.3 DTO AccountRequest

Create the file:

account-module/src/main/java/com/m2ibank/account/dto/AccountRequest.java

```
package com.m2ibank.account.dto;


import com.m2ibank.account.entity.AccountType;
import jakarta.validation.constraints. DecimalMin ;
import jakarta.validation.constraints. NotNull ;


import java.math.BigDecimal;


public class AccountRequest {
```

```
    @NotNull (message = "Customer ID is required" )
    private Long customerId ;

    @NotNull (message = "Account type is required" )
    private AccountType accountType ;

    @NotNull (message = "Initial balance is required" )
     @DecimalMin (value = "0.00" , inclusive = true , message =
"Initial balance must be greater than or equal to zero" )
    private BigDecimal initialBalance ;

    public Long getCustomerId () {
        return customerId ;
}

    public void setCustomerId (Long customerId) {
        this . customerId = customerId;
}

    public AccountType getAccountType () {
        return accountType ;
}

    public void setAccountType (AccountType accountType) {
```

```
        this . accountType = accountType;
}


    public BigDecimal getInitialBalance () {
        return initialBalance ;
}


    public void setInitialBalance (BigDecimal initialBalance) {
        this . initialBalance = initialBalance;
}
}
```

3.5.4 DTO AccountResponse

Create the file:

account-module/src/main/java/com/m2ibank/account/dto/AccountResponse.java

```
package com.m2ibank.account.dto;


import com.m2ibank.account.entity.AccountType;


import java.math.BigDecimal;
import java.time.LocalDateTime;


public class AccountResponse {


    private Long id ;
```

```
   private String accountNumber ;

   private BigDecimal balance ;

   private AccountType accountType ;

   private Long customerId ;

   private LocalDateTime createdAt ;


      public AccountResponse (Long id, String accountNumber,
BigDecimal balance, AccountType accountType, Long customerId,
LocalDateTime createdAt) {

       this.id = id ;

       this . accountNumber = accountNumber;

       this.balance = balance ;

       this . accountType = accountType;

       this . customerId = customerId;

       this.createdAt = createdAt ;

}


   public Long getId () {

       return id ;

}


   public String getAccountNumber () {

       return accountNumber ;

}
```

```
    public BigDecimal getBalance () {
        return balance ;
}

    public AccountType getAccountType () {
        return accountType ;
}

    public Long getCustomerId () {
        return customerId ;
}

    public LocalDateTime getCreatedAt () {
        return createdAt ;
}
}
```

3.5.5 Repository AccountRepository

Create the file:

account-module/src/main/java/com/m2ibank/account/repository/AccountRepository.java

```
package com.m2ibank.account.repository;

import com.m2ibank.account.entity.Account;
```

```
import org.springframework.data.jpa.repository.JpaRepository;


import java.util.List;
import java.util.Optional;


public interface AccountRepository extends
JpaRepository<Account, Long> {


Optional<Account> findByAccountNumber (String accountNumber);
List<Account> findByCustomerId (Long customerId);
}
```

3.5.6 Account Service

Create the file:

account-module/src/main/java/com/m2ibank/account/service/AccountService.java

```
package com.m2ibank.account.service;


import com.m2ibank.account.dto.AccountRequest;
import com.m2ibank.account.dto.AccountResponse;
import com.m2ibank.account.entity.Account;
import com.m2ibank.account.repository.AccountRepository;
import com.m2ibank.common.exception.BusinessException;
import com.m2ibank.common.exception.ResourceNotFoundException;
import com.m2ibank.customer.service.CustomerService;
```

```
import org.springframework.stereotype. Service ;


import java.util.List;

import java.util.UUID;


@Service

public class AccountService {


   private final AccountRepository accountRepository ;

   private final CustomerService customerService ;


    public AccountService (AccountRepository accountRepository,
CustomerService customerService) {

       this . accountRepository = accountRepository;

       this . customerService = customerService;

}


   public AccountResponse createAccount (AccountRequest request)
{

                                                  customerService
.getCustomerEntityById(request.getCustomerId());


String accountNumber = generateAccountNumber();

                                  while     (     accountRepository
.findByAccountNumber(accountNumber).isPresent()) {
```

```
accountNumber = generateAccountNumber();
}

Account account = new Account(
accountNumber,
request.getInitialBalance(),
request.getAccountType(),
request.getCustomerId()
);

Account saved = accountRepository .save(account);
        return mapToResponse(saved);
}

   public AccountResponse getAccountById (Long id) {
        return mapToResponse(getAccountEntityById(id));
}

     public  List<AccountResponse>  getAccountsByCustomerId  (Long
customerId) {
        customerService .getCustomerEntityById(customerId);
        return accountRepository .findByCustomerId(customerId)
.stream()
.map( this ::mapToResponse)
```

```
.toList();

}


   public Account getAccountEntityById (Long id) {

       return accountRepository .findById(id)

.orElseThrow(() -> new ResourceNotFoundException( "Account not
found with id " + id ));

}


           public    void    debitAccount    (Account    account,
java.math.BigDecimal amount) {

       if (account.getBalance().compareTo(amount) < 0 ) {

              throw new BusinessException( "Insufficient balance
for transfer" );

}

account.setBalance(account.getBalance().subtract(amount));

       accountRepository .save(account);

}


           public    void    creditAccount    (Account    account,
java.math.BigDecimal amount) {

account.setBalance(account.getBalance().add(amount));

       accountRepository .save(account);

}
```

```
    private String generateAccountNumber () {
        return "DB-" + UUID. randomUUID ().toString().substring(
0 , 8 ).toUpperCase();
}

    private AccountResponse mapToResponse (Account account) {
        return new AccountResponse(
account.getId(),
account.getAccountNumber(),
account.getBalance(),
account.getAccountType(),
account.getCustomerId(),
account.getCreatedAt()
);
}
}
```

3.5.7 AccountController

Create the file:

account-module/src/main/java/com/m2ibank/account/controller/AccountController.java

```
package com.m2ibank.account.controller;


import com.m2ibank.account.dto.AccountRequest;
import com.m2ibank.account.dto.AccountResponse;
```

```java
import com.m2ibank.account.service.AccountService;

import com.m2ibank.common.api.ApiResponse;

import jakarta.validation. Valid ;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;


import java.util.List;


@RestController

@RequestMapping ( "/api/accounts" )

public class AccountController {


   private final AccountService accountService ;


   public AccountController (AccountService accountService) {

       this . accountService = accountService;

}


   @PostMapping

           public    ResponseEntity<ApiResponse<AccountResponse>>
createAccount ( @Valid @RequestBody AccountRequest request) {

AccountResponse          response          =          accountService
.createAccount(request);

       return ResponseEntity. status ( HttpStatus.CREATED )
```

```
.body(ApiResponse.success ( "Account created successfully" ,
response));

}


   @GetMapping ( "/{id}" )

           public    ResponseEntity<ApiResponse<AccountResponse>>
getAccountById ( @PathVariable Long id) {

AccountResponse response = accountService .getAccountById(id);

        return ResponseEntity. ok (ApiResponse.success ( "Account
retrieved successfully" , response));

}


   @GetMapping ( "/customer/{customerId}" )

      public  ResponseEntity<ApiResponse<List<AccountResponse>>>
getAccountsByCustomerId ( @PathVariable Long customerId) {

List<AccountResponse>       response       =       accountService
.getAccountsByCustomerId(customerId);

             return  ResponseEntity.  ok  (ApiResponse.success  (
"Accounts retrieved successfully" , response));

}

}
```

3.6 Implementation of the transfer-module

The transfer module carries a particularly important part of the business logic, as it embodies one of the most representative uses of a digital banking application: the transfer between accounts.

3.6.1 Transfer Entity

Create the file:

transfer-module/src/main/java/com/m2ibank/transfer/entity/Transfer.java

```
package com.m2ibank.transfer.entity;


import jakarta.persistence.*;


import java.math.BigDecimal;
import java.time.LocalDateTime;


@Entity
@Table (name = "transfers" )
public class Transfer {


   @Id
@GeneratedValue ( strategy = GenerationType.IDENTITY )
   private Long id ;


   @Column (nullable = false )
   private Long sourceAccountId ;


   @Column (nullable = false )
   private Long destinationAccountId ;
```

```
    @Column (nullable = false , precision = 19 , scale = 2 )
    private BigDecimal amount ;

    @Column (length = 255 )
    private String description ;

    @Column (nullable = false )
    private LocalDateTime createdAt ;

    public Transfer () {
        this . createdAt = LocalDateTime. now ();
}

            public    Transfer    (Long    sourceAccountId,    Long
destinationAccountId, BigDecimal amount, String description) {
        this . sourceAccountId = sourceAccountId;
        this . destinationAccountId = destinationAccountId;
        this.amount = amount ;
        this . description = description;
        this . createdAt = LocalDateTime. now ();
}

    public Long getId () {
        return id ;
```

```
}


   public Long getSourceAccountId () {

       return sourceAccountId ;

}


   public Long getDestinationAccountId () {

       return destinationAccountId ;

}


   public BigDecimal getAmount () {

       return amount ;

}


   public String getDescription () {

       return description ;

}


   public LocalDateTime getCreatedAt () {

       return createdAt ;

}

}
```

3.6.2 DTO TransferRequest

Create the file:

transfer-module/src/main/java/com/m2ibank/transfer/dto/TransferRequest.java

```
package com.m2ibank.transfer.dto;


import jakarta.validation.constraints. DecimalMin ;

import jakarta.validation.constraints. NotNull ;

import jakarta.validation.constraints. Size ;


import java.math.BigDecimal;


public class TransferRequest {


   @NotNull (message = "Source account ID is required" )

   private Long sourceAccountId ;


   @NotNull (message = "Destination account ID is required" )

   private Long destinationAccountId ;


   @NotNull (message = "Amount is required" )

    @DecimalMin (value = "0.01" , inclusive = true , message =
"Transfer amount must be greater than zero" )

   private BigDecimal amount ;


   @Size (max = 255 , message = "Description must not exceed 255
characters" )
```

```
   private String description ;


   public Long getSourceAccountId () {
       return sourceAccountId ;
}


   public void setSourceAccountId (Long sourceAccountId) {
       this . sourceAccountId = sourceAccountId;
}


   public Long getDestinationAccountId () {
       return destinationAccountId ;
}


            public    void    setDestinationAccountId    (Long
destinationAccountId) {
       this . destinationAccountId = destinationAccountId;
}


   public BigDecimal getAmount () {
       return amount ;
}


   public void setAmount (BigDecimal amount) {
```

```
        this.amount = amount ;
}


   public String getDescription () {
        return description ;
}


   public void setDescription (String description) {
        this . description = description;
}
}
```

3.6.3 DTO TransferResponse

Create the file:

transfer-module/src/main/java/com/m2ibank/transfer/dto/TransferResponse.java

```
package com.m2ibank.transfer.dto;


import java.math.BigDecimal;
import java.time.LocalDateTime;


public class TransferResponse {


   private Long id ;
   private Long sourceAccountId ;
```

```
   private Long destinationAccountId ;

   private BigDecimal amount ;

   private String description ;

   private LocalDateTime createdAt ;


    public TransferResponse (Long id, Long sourceAccountId, Long
destinationAccountId,  BigDecimal  amount,  String  description,
LocalDateTime createdAt) {

       this.id = id ;

       this . sourceAccountId = sourceAccountId;

       this . destinationAccountId = destinationAccountId;

       this.amount = amount ;

       this . description = description;

       this.createdAt = createdAt ;

}


   public Long getId () {

       return id ;

}


   public Long getSourceAccountId () {

       return sourceAccountId ;

}


   public Long getDestinationAccountId () {
```

```
        return destinationAccountId ;
}


    public BigDecimal getAmount () {
        return amount ;
}


    public String getDescription () {
        return description ;
}


    public LocalDateTime getCreatedAt () {
        return createdAt ;
}
}
```

3.6.4 Repository TransferRepository

Create the file:

transfer-module/src/main/java/com/m2ibank/transfer/repository/TransferRepository.java

```
package com.m2ibank.transfer.repository;


import com.m2ibank.transfer.entity.Transfer;
import org.springframework.data.jpa.repository.JpaRepository;
```

```
import java.util.List;


public interface TransferRepository extends
JpaRepository<Transfer, Long> {


List<Transfer> findBySourceAccountIdOrDestinationAccountId (Long
sourceAccountId, Long destinationAccountId);

}
```

3.6.5 TransferService

Create the file:

transfer-module/src/main/java/com/m2ibank/transfer/service/TransferService.java

```
package com.m2ibank.transfer.service;


import com.m2ibank.account.entity.Account;

import com.m2ibank.account.service.AccountService;

import com.m2ibank.common.exception.BusinessException;

import com.m2ibank.transfer.dto.TransferRequest;

import com.m2ibank.transfer.dto.TransferResponse;

import com.m2ibank.transfer.entity.Transfer;

import com.m2ibank.transfer.repository.TransferRepository;

import org.springframework.stereotype. Service ;

import  org.springframework.transaction.annotation.  Transactional
;
```

```
import java.util.List;


@Service
public class TransferService {


   private final TransferRepository transferRepository ;
   private final AccountService accountService ;


              public     TransferService     (TransferRepository
transferRepository, AccountService accountService) {
       this . transferRepository = transferRepository;
       this . accountService = accountService;
}


   @Transactional
      public  TransferResponse  createTransfer  (TransferRequest
request) {
                                                              if
(request.getSourceAccountId().equals(request.getDestinationAccou
ntId())) {
            throw new BusinessException( "Source and destination
accounts must be different" );
}
```

```
Account sourceAccount = accountService
.getAccountEntityById(request.getSourceAccountId());

Account destinationAccount = accountService
.getAccountEntityById(request.getDestinationAccountId());


                    accountService .debitAccount(sourceAccount,
request.getAmount());

                accountService .creditAccount(destinationAccount,
request.getAmount());


Transfer transfer = new Transfer(

request.getSourceAccountId(),

request.getDestinationAccountId(),

request.getAmount(),

request.getDescription()

);


Transfer saved = transferRepository .save(transfer);

       return mapToResponse(saved);

}


     public List<TransferResponse> getTransfersForAccount (Long
accountId) {

       accountService .getAccountEntityById(accountId);

                                    return     transferRepository
.findBySourceAccountIdOrDestinationAccountId(accountId,
accountId)
```

```
.stream()
.map( this ::mapToResponse)
.toList();
}


   private TransferResponse mapToResponse (Transfer transfer) {
        return new TransferResponse(
transfer.getId(),
transfer.getSourceAccountId(),
transfer.getDestinationAccountId(),
transfer.getAmount(),
transfer.getDescription(),
transfer.getCreatedAt()
);
}
}
```

3.6.6 TransferController

Create the file:

transfer-module/src/main/java/com/m2ibank/transfer/controller/TransferController.java

```
package com.m2ibank.transfer.controller;


import com.m2ibank.common.api.ApiResponse;
```

```
import com.m2ibank.transfer.dto.TransferRequest;

import com.m2ibank.transfer.dto.TransferResponse;

import com.m2ibank.transfer.service.TransferService;

import jakarta.validation. Valid ;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;


import java.util.List;


@RestController

@RequestMapping ( "/api/transfers" )

public class TransferController {


   private final TransferService transferService ;


   public TransferController (TransferService transferService) {

       this . transferService = transferService;

}


   @PostMapping

          public    ResponseEntity<ApiResponse<TransferResponse>>
createTransfer ( @Valid @RequestBody TransferRequest request) {

TransferResponse         response         =         transferService
.createTransfer(request);
```

```
        return ResponseEntity. status ( HttpStatus.CREATED )
.body(ApiResponse.success ( "Transfer executed successfully" ,
response));
}

   @GetMapping ( "/account/{accountId}" )
      public ResponseEntity<ApiResponse<List<TransferResponse>>>
getTransfersForAccount ( @PathVariable Long accountId) {
List<TransferResponse> response = transferService
.getTransfersForAccount(accountId);
             return ResponseEntity. ok (ApiResponse.success (
"Transfers retrieved successfully" , response));
}
}
```

3.7 Implementation of the digibank-web module

The digibank-web module assembles the entire application. It is in this module that the main Spring Boot class and cross-functional configurations are placed.

3.7.1 Main class DigiBankApplication

Create the file:

digibank-web/src/main/java/com/m2ibank/web/DigiBankApplication.java

```
package com.m2ibank.web;

import org.springframework.boot.SpringApplication;
import                    org.springframework.boot.autoconfigure.
SpringBootApplication ;
```

```
import  org.springframework.boot.autoconfigure.domain.  EntityScan
;

import org.springframework.context.annotation. ComponentScan ;

import            org.springframework.data.jpa.repository.config.
EnableJpaRepositories ;


@SpringBootApplication

@ComponentScan (basePackages = "com.m2ibank" )

@EntityScan (basePackages = "com.m2ibank" )

@EnableJpaRepositories (basePackages = "com.m2ibank" )

public class DigiBankApplication {


   public static void main (String[] args) {

SpringApplication. run (DigiBankApplication. class , args);

}

}
```

The attribute scanBasePackages = "com.m2ibank" is crucial here, as it allows Spring to detect the components defined in all functional modules.

3.8 Overall Exception Management

To avoid inconsistent responses or raw errors, a comprehensive exception handler must be implemented. This decision already paves the way for improved application robustness and better control over error messages, an important topic for future security workshops.

Create the file:

digibank-web/src/main/java/com/m2ibank/web/exception/GlobalExceptionHandler.java

```
package com.m2ibank.web.exception;


import com.m2ibank.common.api.ApiResponse;

import com.m2ibank.common.exception.BusinessException;

import com.m2ibank.common.exception.ResourceNotFoundException;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import
org.springframework.web.bind.MethodArgumentNotValidException;

import  org.springframework.web.bind.annotation.  ExceptionHandler
;

import                    org.springframework.web.bind.annotation.
RestControllerAdvice ;


@RestControllerAdvice

public class GlobalExceptionHandler {


   @ExceptionHandler (ResourceNotFoundException. class )

      public  ResponseEntity<ApiResponse<Object>>  handleNotFound
(ResourceNotFoundException ex) {

       return ResponseEntity. status ( HttpStatus.NOT_FOUND )

.body(ApiResponse. failure (ex.getMessage(), null ));

}
```

```
   @ExceptionHandler (BusinessException. class )
                  public      ResponseEntity<ApiResponse<Object>>
handleBusinessException (BusinessException ex) {
       return ResponseEntity. status ( HttpStatus.BAD_REQUEST )
.body(ApiResponse. failure (ex.getMessage(), null ));
}

   @ExceptionHandler (MethodArgumentNotValidException. class )
                  public      ResponseEntity<ApiResponse<Object>>
handleValidationException (MethodArgumentNotValidException ex) {
String message = ex.getBindingResult().getFieldErrors()
.stream()
.findFirst()
.map(error      ->      error.getField()      +      ":      "      +
error.getDefaultMessage())
.orElse( "Validation error" );
       return ResponseEntity. status ( HttpStatus.BAD_REQUEST )
.body(ApiResponse.failure ( message, null ));
}

   @ExceptionHandler (Exception. class )
                  public      ResponseEntity<ApiResponse<Object>>
handleGenericException (Exception ex) {
                        return    ResponseEntity.    status    (
HttpStatus.INTERNAL_SERVER_ERROR )
```

```
.body(ApiResponse.failure("An unexpected error occurred", null));
    }
}
```

3.9 First functional view expected after this part

By the end of this section, the application should already allow:

- To create a client
- To consult a client
- To list the clients
- To create an account for an existing customer
- To view an account
- To list a client's accounts
- To execute a transfer between two accounts
- To view the simple transfer history of an account.

This already provides a credible first functional backbone for DigiBank. This foundation is sufficiently robust to support the rest of the workshop, including Swagger documentation, the index view, datasets, JUnit and Cucumber tests, as well as the preparation for containerization and the CI/CD pipeline.

3.10 Compilation verification at this stage

Once all these files are created, the student must run a global Maven build:

```
mvn clean install
```

Then start the application from the digibank-web module:

```
mvn spring-boot:run -pl digibank-web
```

If everything is configured correctly, Spring Boot should start without errors, connect to PostgreSQL, and expose the REST endpoints defined in the modules. This step constitutes the first serious validation of the application implementation.

Part 4: Advanced application configuration, index view, Swagger/OpenAPI documentation, initialization data, and initial runtime checks

4.1 Role of this phase

At this stage, DigiBank's main business components already exist in the code, but an application doesn't become truly usable simply because its entities, services, and controllers compile. It still needs to be configured, exposed, presented, and validated within a coherent framework.

This fourth part addresses precisely this need. It aims to finalize the implementation framework by adding:

- A better-structured Spring Boot configuration
- A single index view serving as a pedagogical interface
- Swagger/OpenAPI documentation for the exposed APIs
- Initialization data
- Initial comprehensive checks of application functionality.

This logic is consistent with the decision already made for DigiBank: to offer a single simple entry view that presents the application and gives direct access to the API documentation, without transforming the workshop into a complete front-end project.

4.2 Consolidation of the application.yml file

The first task is to complete the central configuration of the digibank-web module. The digibank-web/src/main/resources/application.yml file must be enhanced to improve local execution, log readability, database access, and API documentation.

Replace the previous content with:

```
spring :
 application :
   name : digibank


 datasource :
   url : jdbc:postgresql://localhost:5432/digibankdb
   username : digibank
   password : digibank123
   driver-class-name : org.postgresql.Driver


 jpa :
   hibernate :
     ddl-auto : update
   show-sql : true
   open-in-view : false
   properties :
```

```
    hibernate :
      SQL format : true
      jdbc :
        time zone : UTC

 thymeleaf :
   cache : false

 SQL :
   init :
     mode : never

server :
 port : 8080
 error :
   include-message : never
   include-binding-errors : never
   include-stacktrace : never

springdoc :
 api-docs :
   path : /api-docs
 swagger-ui :
   path : /swagger-ui.html
```

```
   operationsSorter : method
   tagsSorter : alpha


logging :
 level :
   root : INFO
   org.springframework.web : INFO
   org.hibernate.SQL : DEBUG
   org.hibernate.type.descriptor.sql.BasicBinder : TRACE
```

This configuration maintains a simple pedagogical approach, while already improving several important points: activation of API documentation, better SQL readability, deactivation of the Thymeleaf cache during development and limitation of certain technical information in case of error.

It is also important to explain to the students that this configuration is not yet a production environment. Basic credentials are still visible for pedagogical reasons, explicitly preparing them for future workshops focused on securing secrets and deployment.

4.3 Setting up Spring profiles

Even in a first implementation workshop, it is useful to introduce an initial separation by user profiles. This shows students that an application does not necessarily have to be configured identically in all contexts.

Create an application-dev.yml file:

digibank-web/src/main/resources/application-dev.yml

```
spring :
 datasource :
```

```
  url : jdbc:postgresql://localhost:5432/digibankdb
  username : digibank
  password : digibank123
  driver-class-name : org.postgresql.Driver

jpa :
  hibernate :
    ddl-auto : update
```

Also create an application-test.yml file:

digibank-web/src/main/resources/application-test.yml

```
spring :
 datasource :
  url : jdbc:h2:mem:digibanktestdb
  driver-class-name : org.h2.Driver
  username : sa
  password :

 jpa :
  hibernate :
    ddl-auto : create-drop
  show-sql : false
```

For this test profile to work, H2 will later need to be added to the test dependencies of the digibank-web module. This preparation is important for what follows, particularly for integration testing and pipeline automation.

In the application.yml file, you can define the default active profile:

```
spring :
 profiles :
   active : dev
```

The new content of application.yml therefore becomes

```
spring :
 application :
   name : digibank


 jpa :
   hibernate :
     ddl-auto : update
   show-sql : true
   open-in-view : false
   properties :
     hibernate :
       SQL format : true
       jdbc :
         time zone : UTC


 thymeleaf :
```

```
  cache : false


 SQL :
   init :
     mode : never


server :
 port : 8080
 error :
   include-message : never
   include-binding-errors : never
   include-stacktrace : never


springdoc :
 api-docs :
   path : /api-docs
 swagger-ui :
   path : /swagger-ui.html
   operationsSorter : method
   tagsSorter : alpha


logging :
 level :
   root : INFO
```

```
  org.springframework.web : INFO

  org.hibernate.SQL : DEBUG

  org.hibernate.type.descriptor.sql.BasicBinder : TRACE
```

This practice introduces a good industrialization reflex early on.

4. DigiBank Single Index View

As decided for the project, the application must have a single index view presenting DigiBank and providing a link to the Swagger documentation. This decision is pedagogical: it gives the application a simple interface without diverting students into overly complex front-end development.

4.4.1 Creating the view controller

Create the file:

digibank-web/src/main/java/com/m2ibank/web/controller/HomeController.java

```
package com.m2ibank.web.controller;


import org.springframework.stereotype. Controller ;

import org.springframework.ui.Model;

import org.springframework.web.bind.annotation. GetMapping ;


@Controller

public class HomeController {


  @GetMapping ( "/" )
```

```
   public String home (Model model) {

model.addAttribute( "applicationName" , "DigiBank" );

model.addAttribute(  "applicationDescription"  ,  "Digital  Core
Banking System for M2iBank" );

model.addAttribute( "swaggerUrl" , "/swagger-ui.html" );

       return "index" ;

}

}
```

This controller is intentionally simple. Its role is not to carry business logic, but to provide a minimal, clean, and consistent homepage that aligns with the pedagogical objective of the practical exercise.

4.4.2 Creating the index.html template

Create the file:

digibank-web/src/main/resources/templates/index.html

```
<!DOCTYPE html >

<html lang ="en" >

<head>

<meta charset ="UTF-8" >

<title> DigiBank - Home </title>

<meta     name     ="viewport"     content     ="width=device-width,
initial-scale=1.0" >

<style>

       body {
```

```
font-family: Arial, sans-serif;
background: #f4f7fb;
color: #1e293b;
margin: 0;
padding: 0;
}

.container {
max-width: 980px;
margin: 60px auto;
background: white;
border-radius: 16px;
padding: 40px;
box-shadow: 0 8px 24px rgba(0,0,0,0.08);
}

h1 {
color: #0f172a;
margin-bottom: 10px;
}

p {
line-height: 1.6;
margin-bottom: 16px;
```

```
}

.badge {
display: inline-block;
background: #0ea5e9;
color: white;
padding: 8px 14px;
border-radius: 999px;
font-size: 14px;
margin-bottom: 24px;
}

.section {
margin-top: 30px;
}

.button {
display: inline-block;
text-decoration: none;
background: #2563eb;
color: white;
padding: 12px 20px;
border-radius: 8px;
font-weight: bold;
```

```
transition: background 0.2s ease-in-out;
}

.button:hover {
background: #1d4ed8;
}

ul {
padding-left: 20px;
}

li {
margin-bottom: 10px;
}

footer {
margin-top: 40px;
font-size: 14px;
color: #64748b;
}
   </style>
</head>
<body>
<div class ="container" >
```

```
<div class ="badge" > M2iBank - Digital Core Banking System
</div>


<h1 th :text ="${applicationName}" > DigiBank </h1>


<p th :text ="${applicationDescription}" >

       Digital Core Banking System for M2iBank

   </p>


<div class ="section" >

<h2> Project Overview </h2>

<p>

            DigiBank is a modular monolithic banking application
developed as part of the DevSecOps workshop.

It provides core services for customer management, account
management and bank transfers.

       </p>

</div>


<div class ="section" >

<h2> Available Functional Domains </h2>

<ul>

<li> Customer registration and consultation. </li>

<li> Account creation and account listing by customer. </li>

<li> Transfer execution between accounts. </li>
```

```
<li> Transfer history consultation for a specific account. </li>

</ul>

</div>


<div class ="section" >

<h2> API Documentation </h2>

<p>

                The REST API documentation is available through
Swagger/OpenAPI.

       </p>

<a    class    ="button"    th    :href    ="${swaggerUrl}"    href
="/swagger-ui.html" > Open Swagger UI </a>

</div>


<footer>

          DigiBank - Workshop 1 - M2iBank - DevSecOps Learning
Project

   </footer>

</div>

</body>

</html>
```

This view fulfills exactly the expected function: it presents the application, outlines its scope, and provides explicit access to the API documentation.

4.5 Swagger/OpenAPI Documentation

The addition of Swagger is essential in this workshop. It allows students to visualize exposed endpoints, understand the structure of requests and responses, and then quickly experiment with APIs without immediately writing a complete client.

4.5.1 Swagger Dependency Check

The following dependency must already be present in digibank-web/pom.xml:

```
<dependency>
<groupId> org.springdoc </groupId>
<artifactId> springdoc-openapi-starter-webmvc-ui </artifactId>
<version> ${springdoc.version} </version>
</dependency>
```

If it has not yet been added, it must be integrated and then the Maven project reloaded in IntelliJ IDEA.

4.5.2 OpenAPI configuration class

Create the file:

digibank-web/src/main/java/com/m2ibank/web/config/OpenApiConfig.java

```
package com.m2ibank.web.config;


import io.swagger.v3.oas.models.ExternalDocumentation;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Contact;
import io.swagger.v3.oas.models.info.Info;
```

```
import io.swagger.v3.oas.models.info.License;

import org.springframework.context.annotation. Bean ;

import org.springframework.context.annotation. Setup ;


@Configuration

public class OpenApiConfig {


   @Bean

   public OpenAPI digiBankOpenAPI () {

       return new OpenAPI()

.info( new Info()

.title( "DigiBank API " )

.description( "REST API documentation for the DigiBank modular
monolith application" )

.version( "1.0.0" )

.contact( new Contact()

.name( "M2iBank DevSecOps Workshop" )

.email( "contact@m2ibank.local" ))

.license( new License()

.name( "Educational Use Only" )

.url( "https://example.org/license" )))

.externalDocs( new ExternalDocumentation()

.description( "DigiBank Project Documentation" )

.url( "https://example.org/digibank" ));
```

```
}

}
```

This class structures the documentation exposed by Swagger UI. Even though the example URLs here are for educational purposes, it already demonstrates how to properly enhance a documented API.

4.5.3 Controller Documentation

It is useful to add some Swagger annotations to controllers to improve the readability of endpoints in the interface. For example, in CustomerController, you can enhance the class:

```
import io.swagger.v3.oas.annotations.Operation ;

import io.swagger.v3.oas.annotations.tags.Tag ;
```

Then complete the controller:

```
@ Tag ( name = "Customers" , description = "Customer management
API" )

@RestController

@RequestMapping ( “/api/customers ” )

audience class CustomerController {
```

And document the methods:

```
@ Operation ( summary) = "Create a new customer "

@ PostMapping
```

```
audience ResponseEntity < ApiResponse <CustomerResponse>
createCustomer (@ Valid @ RequestBody CustomerRequest request) {

...

}
```

The same logic can be applied to AccountController and TransferController. This improvement is not essential at the outset, but it enhances the educational value of the project and the readability of the API documentation.

4.6 Demonstration Data Initialization

An empty banking application is not very informative during initial demonstrations. It is therefore useful to initialize some basic data to allow for quick manual testing in Swagger or Postman.

4.6.1 Creating a CommandLineRunner

Create the file:

digibank-web/src/main/java/com/m2ibank/web/bootstrap/DataInitializer.java

```
package com.m2ibank.web.bootstrap;


import com.m2ibank.account.dto.AccountRequest;
import com.m2ibank.account.entity.AccountType;
import com.m2ibank.account.service.AccountService;
import com.m2ibank.customer.dto.CustomerRequest;
import com.m2ibank.customer.dto.CustomerResponse;
import com.m2ibank.customer.service.CustomerService;
```

```
import org.springframework.boot.CommandLineRunner;

import org.springframework.stereotype. Component ;


import java.math.BigDecimal;


@Component

public class DataInitializer implements CommandLineRunner {


   private final CustomerService customerService ;

   private final AccountService accountService ;


      public  DataInitializer  (CustomerService  customerService,
AccountService accountService) {

       this . customerService = customerService;

       this . accountService = accountService;

}


   @Override

   public void run (String...args) {

       if ( customerService.getAllCustomers ().isEmpty()) {

CustomerRequest customer1 = new CustomerRequest();

customer1.setFullName( "Alice Ndzi" );

customer1.setEmail( "alice@m2ibank.com" );

customer1.setPhoneNumber( "+237600000001" );
```

```
customer1.setNationalId( "CNI000001" );


CustomerRequest customer2 = new CustomerRequest();

customer2.setFullName( "Brian Tchoumi" );

customer2.setEmail( "brian@m2ibank.com" );

customer2.setPhoneNumber( "+237600000002" );

customer2.setNationalId( "CNI000002" );


CustomerResponse      savedCustomer1      =      customerService
.createCustomer(customer1);

CustomerResponse      savedCustomer2      =      customerService
.createCustomer(customer2);


AccountRequest account1 = new AccountRequest();

account1.setCustomerId(savedCustomer1.getId());

account1.setAccountType(AccountType.CURRENT ) ;

account1.setInitialBalance( new BigDecimal( "150000.00" ));


AccountRequest account2 = new AccountRequest();

account2.setCustomerId(savedCustomer2.getId());

account2.setAccountType(AccountType.SAVINGS ) ;

account2.setInitialBalance( new BigDecimal( "90000.00" ));


          accountService .createAccount(account1);

          accountService .createAccount(account2);
```

```
}

}

}
```

This class automatically populates the database at startup if no clients yet exist. It makes initial demonstrations much smoother, as the student immediately has concrete examples to work with.

It should be noted, however, that this strategy is intended for development or demonstration purposes. In production environments, more controlled migration and seeding mechanisms would be preferable.

4.7 Adding H2 for testing

As a test profile was provided above, H2 must now be added to the digibank-web module. In digibank-web/pom.xml, add the following dependency:

```
<dependency>

<groupId> org.springframework.boot </groupId>

<artifactId> spring-boot-starter-test </artifactId>

<scope> test </scope>

</dependency>
```

This dependency will facilitate automated testing without systematically requiring a real PostgreSQL instance. This is a common best practice in educational Spring Boot projects and prepares the logical continuation of the workshop.

4.8 Initial execution checks

Once the configuration files, index view, Swagger, and data initialization are in place, the student must restart the application:

```
mvn clean install

mvn spring-boot:run -pl digibank-web
```

If everything is correct, several checks must be carried out.

4.8.1 Homepage Check

In the browser, open:

```
http : //localhost:8080/
```

The DigiBank homepage should display with:

- The name of the application
- A brief description
- A reminder of the functional areas
- A button to access the Swagger UI.

4.8.2 Swagger UI Verification

Open :

```
http :// localhost : 8080 / swagger - ui . html
```

The Swagger interface should display the APIs:

- Customers
- Accounts
- Transfers

The student must verify that he/she can:

- View query patterns
- View exposed endpoints
- Making simple calls from the interface
- Retrieve the normalized JSON responses.

4.8.3 Verification of initial data

Using Swagger or an HTTP client, the student must verify the following routes:

```
GET /api/customers

GET /api/accounts/customer/{customerId}
```

The goal is to confirm that the demo clients and accounts have been created automatically.

4.8.4 Testing a first transfer

The student can then call:

```
POST /api/transfers
```

With a JSON body of the type:

```
{
 "sourceAccountId" : 1 ,
 "destinationAccountId" : 2 ,
 "amount " : 5000.00
 "description" : "Initial demo transfer"
}
```

Then check:

- That the answer indicates success
- The transfer has been successfully completed.
- That the credit has been successfully processed
- The account history should correctly reflect the transfer.

4.9 Safety adjustment already visible at this stage

Even though Workshop 1 isn't yet a SAST or DAST workshop, it's useful to establish some good habits at this stage. The server configuration has already begun to reduce some leaks of technical information in error responses, which naturally prepares the ground for future security workshops.

Students need to be reminded that:

- Error messages should not expose the entire internal stack
- The basic identifiers visible in application.yml are purely educational.
- Validating entries at the DTO level is already a first step towards strengthening security.
- Consistency in API responses facilitates testing, maintenance, and future analysis.

4.10 Expected state of the project at the end of this part

By the end of this fourth part, DigiBank must be a truly presentable and demonstrable application. It must offer:

- A stable Spring Boot execution
- An effective connection to PostgreSQL
- An accessible index view
- A functional Swagger documentation
- Automatic initialization data
- End-to-end testable REST endpoints.

In other words, the project is no longer just a set of compilable classes. It becomes a first version that is visible, navigable and manipulable, which is exactly the pedagogical objective before moving on to more complete automated testing.

Part 5: JUnit Unit Tests, Initial Integration Tests, Cucumber Scenarios, Validation Commands, and Preparing Execution Proofs

5.1 Role of tests in DigiBank

In a project like DigiBank, testing should not be seen as an afterthought, but as an integral part of system quality. Even an educational banking application handles sensitive business rules: customer uniqueness, account creation, initial balance, debits, credits, the inability to transfer funds to the same account, and the inability to transfer funds beyond the available balance. Each of these rules must be demonstrable through explicit testing.

In the chosen architecture, testing serves a dual purpose. JUnit unit tests are used to verify business logic in isolation, primarily within services. Initial integration tests verify that multiple components cooperate correctly in a real Spring Boot environment, while Cucumber scenarios are used to express business use cases in language close to the behavior expected by a user or functional analyst.

This complementarity is pedagogically important because it shows students that the same system can be verified at several levels: the classroom, the collaboration between components, and the overall business scenario.

5.2 Test dependencies to check

Before writing the tests, it is necessary to ensure that the necessary dependencies are present in the modules concerned.

5.2.1 JUnit and Mockito Dependencies

In business modules such as customer-module, account-module, and transfer-module, the following dependencies must already exist or be completed in the pom.xml files:

```
<dependency>
<groupId> org.springframework.boot </groupId>
<artifactId> spring-boot-starter-test </artifactId>
<scope> test </scope>
</dependency>

<dependency>
<groupId> org.mockito </groupId>
<artifactId> mockito-core </artifactId>
<scope> test </scope>
</dependency>
```

For improved comfort with JUnit 5, you can also add:

```
<dependency>
```

```
<groupId> org.mockito </groupId>
<artifactId> mockito-junit-jupiter </artifactId>
<scope> test </scope>
</dependency>
```

These dependencies allow you to isolate business services and simulate dependencies such as repositories or other services.

5.2.2 Cucumber Dependencies

In the digibank-web module, which will serve as the entry point for integration tests and functional scenarios, add the following dependencies if they are not already present:

```
<dependency>
<groupId>io.cucumber</groupId>
<artifactId>cucumber-java</artifactId>
<version>${cucumber.version}</version>
<scope>test</scope>
</dependency>

<dependency>
<groupId>io.cucumber</groupId>
<artifactId>cucumber-spring</artifactId>
<version>${cucumber.version}</version>
<scope>test</scope>
</dependency>

<dependency>
```

```
<groupId>io.cucumber</groupId>

<artifactId>cucumber-junit-platform-engine</artifactId>

<version>${cucumber.version}</version>

<scope>test</scope>

</dependency>


<dependency>

<groupId>org.junit.platform</groupId>

<artifactId>junit-platform-suite</artifactId>

<scope>test</scope>

</dependency>
```

These dependencies will allow business scenarios to be executed directly against a Spring Boot context.

5.3 Selected Testing Strategy

DigiBank's testing strategy must be clear and explicitly explained to students.

It is based on three levels:

- **Unit tests** to validate the business logic of an isolated service
- **Spring Boot integration tests** to verify cooperation between layers
- **Cucumber tests** to validate understandable and traceable functional scenarios.

Unit tests will be placed within the business modules, as each module must have its own logic and verification process. Cucumber tests, on the other hand, will more naturally be centralized in digibank-web, since they focus on the overall behavior of the assembled application.

5.4 Unit tests of the customer module

The first service to test is CustomerService, because it carries simple but important rules: creation of a customer, uniqueness check by email and by phone, retrieval of an existing customer.

5.4.1 Creating the CustomerServiceTest

Create the file:

customer-module/src/test/java/com/m2ibank/customer/service/CustomerServiceTest.java

```
package com.m2ibank.customer.service;


import com.m2ibank.common.exception.BusinessException;
import com.m2ibank.common.exception.ResourceNotFoundException;
import com.m2ibank.customer.dto.CustomerRequest;
import com.m2ibank.customer.dto.CustomerResponse;
import com.m2ibank.customer.entity.Customer;
import com.m2ibank.customer.repository.CustomerRepository;
import org.junit.jupiter.api. BeforeEach ;
import org.junit.jupiter.api. Test ;
import org.junit.jupiter.api.extension. ExtendWith ;
import org.mockito. InjectMocks ;
import org.mockito. Mock ;
import org.mockito.junit.jupiter.MockitoExtension;


import java.util.Optional;
```

```java
import java.util.List;


import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;


@ExtendWith (MockitoExtension. class )
class CustomerServiceTest {


   @Mock
   private CustomerRepository customerRepository ;


   @InjectMocks
   private CustomerService customerService ;


   private CustomerRequest request ;
   private Customer ;


   @BeforeEach
   void setUp () {
       request = new CustomerRequest();
       request .setFullName( "Alice Ndzi" );
       request .setEmail( "alice@m2ibank.com" );
       request .setPhoneNumber( "+237600000001" );
       request .setNationalId( "CNI000001" );
```

```
        customer = new Customer(
                "Alice Ndzi" ,
                "alice@m2ibank.com" ,
                "+237600000001" ,
                "CNI000001"
        );
}


   @Test
   void shouldCreateCustomerSuccessfully () {
        when ( customerRepository.findByEmail ( request.getEmail
()) ).thenReturn(Optional.empty ( ));
        when ( customerRepository .findByPhoneNumber( request
.getPhoneNumber())).thenReturn(Optional. empty ());
        when ( customerRepository .save( any (Customer. class
))).thenAnswer(invocation -> {
Customer saved = invocation.getArgument( 0 );
java.lang.reflect.Field idField = Customer. class
.getDeclaredField( "id" );
idField.setAccessible( true );
idField.set(saved, 1L );
            return saved;
});
```

```
CustomerResponse response = customerService.createCustomer (
request );


        assertNotNull (response);

        assertEquals ( "Alice Ndzi" , response.getFullName());

        assertEquals ( "alice@m2ibank.com" ,
response.getEmail());

        verify ( customerRepository , times ( 1 )).save( any
(Customer. class ));

}


    @Test

    void shouldThrowExceptionWhenEmailAlreadyExists () {

        when ( customerRepository .findByEmail( request
.getEmail())).thenReturn(Optional. of ( customer ));


BusinessException exception = assertThrows (BusinessException.
class ,

() -> customerService.createCustomer ( request ));


        assertEquals ( "A customer with this email already
exists" , exception.getMessage());

        verify ( customerRepository , never ()).save( any
(Customer. class ));

}


    @Test
```

```
   void shouldThrowExceptionWhenPhoneNumberAlreadyExists () {

       when ( customerRepository.findByEmail ( request.getEmail
()) ).thenReturn(Optional.empty ( ));

       when ( customerRepository .findByPhoneNumber( request
.getPhoneNumber())).thenReturn(Optional. of ( customer ));


BusinessException exception = assertThrows (BusinessException.
class ,

() -> customerService.createCustomer ( request ));


       assertEquals ( "A customer with this phone number already
exists" , exception.getMessage());

       verify ( customerRepository , never ()).save( any
(Customer. class ));

}


   @Test

   void shouldReturnCustomerById () throws Exception {

java.lang.reflect.Field idField = Customer. class
.getDeclaredField( "id" );

idField.setAccessible( true );

idField.set( customer , 1L );


       when ( customerRepository.findById ( 1L
)).thenReturn(Optional.of ( customer ) );
```

```
CustomerResponse response = customerService.getCustomerById ( 1L
);


        assertEquals ( 1L , response.getId());

        assertEquals ( "Alice Ndzi" , response.getFullName());

}


    @Test

    void shouldThrowExceptionWhenCustomerNotFound () {

        when ( customerRepository.findById ( 99L
)).thenReturn(Optional.empty ( ));


ResourceNotFoundException exception = assertThrows
(ResourceNotFoundException. class ,

() -> customerService.getCustomerById ( 99L ));


        assertTrue (exception.getMessage().contains( "Customer
not found" ));

}


    @Test

    void shouldReturnAllCustomers () throws Exception {

java.lang.reflect.Field idField = Customer. class
.getDeclaredField( "id" );

idField.setAccessible( true );

idField.set( customer , 1L );
```

```
        when ( customerRepository .findAll()).thenReturn(List. of
( customer ));


List<CustomerResponse> customers = customerService
.getAllCustomers();


        assertEquals ( 1 , customers.size());
        assertEquals ( "Alice Ndzi" , customers.get( 0
).getFullName());
}
}
```

This test already covers the main business rules of the client module. It also shows students how to use Mockito to isolate a service from its persistence layer.

5.5 Unit tests of the account-module

The account module must check several behaviors: creation of an account for an existing customer, recovery of an account, control of debit and credit.

5.5.1 Creating the AccountServiceTest

Create the file:

account-module/src/test/java/com/m2ibank/account/service/AccountServiceTest.java

```
package com.m2ibank.account.service;


import com.m2ibank.account.dto.AccountRequest;
```

```
import com.m2ibank.account.dto.AccountResponse;

import com.m2ibank.account.entity.Account;

import com.m2ibank.account.entity.AccountType;

import com.m2ibank.account.repository.AccountRepository;

import com.m2ibank.common.exception.BusinessException;

import com.m2ibank.common.exception.ResourceNotFoundException;

import com.m2ibank.customer.entity.Customer;

import com.m2ibank.customer.service.CustomerService;

import org.junit.jupiter.api. BeforeEach ;

import org.junit.jupiter.api. Test ;

import org.junit.jupiter.api.extension. ExtendWith ;

import org.mockito. InjectMocks ;

import org.mockito. Mock ;

import org.mockito.junit.jupiter.MockitoExtension;


import java.math.BigDecimal;

import java.util.Optional;

import java.util.List;


import static org.junit.jupiter.api.Assertions.*;

import static org.mockito.Mockito.*;


@ExtendWith (MockitoExtension. class )

class AccountServiceTest {
```

```
    @Mock
    private AccountRepository accountRepository ;

    @Mock
    private CustomerService customerService ;

    @InjectMocks
    private AccountService accountService ;

    private AccountRequest request ;
    private Customer ;
    private account ;

    @BeforeEach
    void setUp () {
        request = new AccountRequest();
        request.setCustomerId ( 1L );
        request .setAccountType(AccountType.CURRENT ) ;
        request .setInitialBalance( new BigDecimal( "100000.00"
));

        customer = new Customer( "Alice Ndzi" ,
"alice@m2ibank.com" , "+237600000001" , "CNI000001" );
```

```
        account = new Account( "DB-ABC12345" , new BigDecimal(
"100000.00" ), AccountType. CURRENT , 1L );

}


    @Test

    void shouldCreateAccountSuccessfully () {

        when ( customerService .getCustomerEntityById( 1L
)).thenReturn( customer );

        when ( accountRepository .findByAccountNumber( anyString
())).thenReturn(Optional. empty ());

        when ( accountRepository .save( any (Account. class
))).thenAnswer(invocation -> {

Account saved = invocation.getArgument( 0 );

java.lang.reflect.Field idField = Account. class
.getDeclaredField( "id" );

idField.setAccessible( true );

idField.set(saved, 1L );

            return saved;

});


AccountResponse response = accountService.createAccount (
request );


        assertNotNull (response);

        assertEquals (AccountType.CURRENT ,
response.getAccountType ());
```

```
        assertEquals ( new BigDecimal( "100000.00" ),
response.getBalance());

        verify ( accountRepository , times ( 1 )).save( any
(Account. class ));

}


    @Test

    void shouldReturnAccountById () throws Exception {

java.lang.reflect.Field idField = Account. class
.getDeclaredField( "id" );

idField.setAccessible( true );

idField.set( account , 1L );


        when ( accountRepository .findById( 1L
)).thenReturn(Optional. of ( account ));


AccountResponse response = accountService.getAccountById ( 1L );


        assertEquals ( 1L , response.getId());

        assertEquals ( "DB-ABC12345" ,
response.getAccountNumber());

}


    @Test

    void shouldThrowExceptionWhenAccountNotFound () {
```

```
        when ( accountRepository.findById ( 99L
)).thenReturn(Optional.empty ( ));


        assertThrows (ResourceNotFoundException. class ,
() -> accountService.getAccountById ( 99L ));
}


    @Test
    void shouldDebitAccountSuccessfully () {
        when ( accountRepository .save( any (Account. class
))).thenReturn( account );


        accountService .debitAccount( account , new BigDecimal(
"5000.00" ));


        assertEquals ( new BigDecimal( "95000.00" ), account
.getBalance());
}


    @Test
    void shouldThrowExceptionWhenBalanceIsInsufficient () {
BusinessException exception = assertThrows (BusinessException.
class ,
() -> accountService .debitAccount( account , new BigDecimal(
"200000.00" )));
```

```
        assertEquals ( "Insufficient balance for transfer" ,
exception.getMessage());
}


    @Test
    void shouldCreditAccountSuccessfully () {
        when ( accountRepository .save( any (Account. class
))).thenReturn( account );


        accountService .creditAccount( account , new BigDecimal(
"5000.00" ));


        assertEquals ( new BigDecimal( "105000.00" ), account
.getBalance());
}


    @Test
    void shouldReturnAccountsByCustomerId () {
        when ( customerService .getCustomerEntityById( 1L
)).thenReturn( customer );
        when ( accountRepository .findByCustomerId( 1L
)).thenReturn(List. of ( account ));


List<AccountResponse> accounts =
accountService.getAccountsByCustomerId ( 1L );


        assertEquals ( 1 ,accounts.size());
```

```
        assertEquals ( 1L , accounts.get( 0 ).getCustomerId());
}
}
```

5.6 Unit tests of the transfer module

The transfer module contains the most sensitive business rules of the first functional set. At a minimum, the following must be verified:

- A transfer cannot be executed between the same source and destination account.
- That a valid transfer debits and credits correctly;
- That the transfer history of an account can be retrieved.

5.6.1 Creating the TransferServiceTest

Create the file:

transfer-module/src/test/java/com/m2ibank/transfer/service/TransferServiceTest.java

```
package com.m2ibank.transfer.service;


import com.m2ibank.account.entity.Account;
import com.m2ibank.account.entity.AccountType;
import com.m2ibank.account.service.AccountService;
import com.m2ibank.common.exception.BusinessException;
import com.m2ibank.transfer.dto.TransferRequest;
import com.m2ibank.transfer.dto.TransferResponse;
import com.m2ibank.transfer.entity.Transfer;
import com.m2ibank.transfer.repository.TransferRepository;
import org.junit.jupiter.api. BeforeEach ;
```

```
import org.junit.jupiter.api. Test ;
import org.junit.jupiter.api.extension. ExtendWith ;
import org.mockito. InjectMocks ;
import org.mockito. Mock ;
import org.mockito.junit.jupiter.MockitoExtension;

import java.math.BigDecimal;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith (MockitoExtension. class )
class TransferServiceTest {

   @Mock
   private TransferRepository transferRepository ;

   @Mock
   private AccountService accountService ;

   @InjectMocks
   private TransferService transferService ;
```

```
    private TransferRequest request ;

    private Account sourceAccount ;

    private Account destinationAccount ;


    @BeforeEach

    void setUp () {

        request = new TransferRequest();

        request .setSourceAccountId( 1L );

        request .setDestinationAccountId( 2L );

        request .setAmount( new BigDecimal( "5000.00" ));

        request .setDescription( "Monthly transfer" );


        sourceAccount = new Account( "DB-SRC12345" , new
BigDecimal( "100000.00" ), AccountType. CURRENT , 1L );

        destinationAccount = new Account( "DB-DST12345" , new
BigDecimal( "50000.00" ), AccountType. SAVINGS , 2L );

}


    @Test

    void shouldCreateTransferSuccessfully () {

        when ( accountService .getAccountEntityById( 1L
)).thenReturn( sourceAccount );

        when ( accountService .getAccountEntityById( 2L
)).thenReturn( destinationAccount );

        doNothing ().when( accountService ).debitAccount(
sourceAccount , new BigDecimal( "5000.00" ));
```

```
        doNothing ().when( accountService ).creditAccount(
destinationAccount , new BigDecimal( "5000.00" ));

        when ( transferRepository .save( any (Transfer. class
))).thenAnswer(invocation -> {

Transfer saved = invocation.getArgument( 0 );

java.lang.reflect.Field idField = Transfer. class
.getDeclaredField( "id" );

idField.setAccessible( true );

idField.set(saved, 1L );

            return saved;

});


TransferResponse response = transferService.createTransfer (
request );


        assertNotNull (response);

        assertEquals ( 1L , response.getSourceAccountId());

        assertEquals ( 2L , response.getDestinationAccountId());

        assertEquals ( new BigDecimal( "5000.00" ),
response.getAmount());

        verify ( accountService , times ( 1 )).debitAccount(
sourceAccount , new BigDecimal( "5000.00" ));

        verify ( accountService , times ( 1 )).creditAccount(
destinationAccount , new BigDecimal( "5000.00" ));

}


    @Test
```

```
   void shouldThrowExceptionWhenSourceAndDestinationAreSame () {

       request .setDestinationAccountId( 1L );


BusinessException exception = assertThrows (BusinessException.
class ,

() -> transferService.createTransfer ( request ));


       assertEquals ( "Source and destination accounts must be
different" , exception.getMessage());

}


   @Test

   void shouldReturnTransfersForAccount () {

Transfer transfer = new Transfer( 1L , 2L , new BigDecimal(
"5000.00" ), "Monthly transfer" );

       when ( accountService .getAccountEntityById( 1L
)).thenReturn( sourceAccount );

       when ( transferRepository
.findBySourceAccountIdOrDestinationAccountId( 1L , 1L ))

.thenReturn(List. of (transfer));


List<TransferResponse> transfers = transferService
.getTransfersForAccount( 1L );


       assertEquals ( 1 , transfers.size());

       assertEquals ( new BigDecimal( "5000.00" ),
transfers.get( 0 ).getAmount());
```

```
}

}
```

5.7 Initial Spring Boot Integration Tests

Unit tests validate isolated rules, but they don't yet prove that the controllers, services, repositories, and Spring configuration work well together. Therefore, at least one initial representative integration test must be added.

5.7.1 Added H2 for testing

In digibank-web/pom.xml, verify that H2 is present:

```
<dependency>

<groupId> com.h2database </groupId>

<artifactId> h2 </artifactId>

<scope> test </scope>

</dependency>
```

5.7.2 Creating the CustomerIntegrationTest

Create the file:

digibank-web/src/test/java/com/m2ibank/web/integration/CustomerIntegrationTest.java

```
package com.m2ibank.web.integration;


import com.fasterxml.jackson.databind.ObjectMapper;

import com.m2ibank.customer.dto.CustomerRequest;
```

```
import org.junit.jupiter.api. DisplayName ;

import org.junit.jupiter.api. Test ;

import org.springframework.beans.factory.annotation. Autowired ;

import org.springframework.boot.test.autoconfigure.web.servlet.
AutoConfigureMockMvc ;

import org.springframework.boot.test.context. SpringBootTest ;

import org.springframework.http.MediaType;

import org.springframework.test.context. ActiveProfiles ;

import org.springframework.test.web.servlet.MockMvc;


import static
org.springframework.test.web.servlet.request.MockMvcRequestBuild
ers. post ;

import static
org.springframework.test.web.servlet.result.MockMvcResultMatcher
s.*;


@SpringBootTest

@AutoConfigureMockMvc

@ActiveProfiles ( "test" )

class CustomerIntegrationTest {


   @Autowired

   private MockMvc mockMvc ;


   @Autowired
```

```
   private ObjectMapper objectMapper ;


   @Test
@DisplayName ( "Should create a customer through REST API" )
   void shouldCreateCustomerThroughApi () throws Exception {
CustomerRequest request = new CustomerRequest();
request.setFullName( "Integration User" );
request.setEmail( "integration@m2ibank.com" );
request.setPhoneNumber( "+237611111111" );
request.setNationalId( "CNI-INTEG-001" );


       mockMvc .perform( post ( "/api/customers" )
.contentType( MediaType.APPLICATION_JSON )
.content( objectMapper .writeValueAsString(request)))
.andExpect( status ().isCreated())
.andExpect( jsonPath ( "$.success" ).value( true ))
.andExpect( jsonPath ( "$.data.fullName" ).value( "Integration
User" ))
.andExpect( jsonPath ( "$.data.email" ).value(
"integration@m2ibank.com" ));
}
}
```

This test already shows how to verify the system through its HTTP API, which is an important step before moving on to richer Cucumber scenarios.

5.8 Setting up Cucumber

Cucumber plays a significant educational role in DigiBank, as it allows functionalities to be expressed in a readable way, closely aligned with business use cases. In a banking application, this makes perfect sense, as rules can be reformulated into scenarios understandable by non-developers.

5.8.1 Recommended Cucumber Tree Structure

In digibank-web, create the following directory structure:

```
src/test/
├── java/
│   └── com/m2ibank/web/cucumber/
│       ├── CucumberSpringConfiguration.java
│       ├── RunCucumberTest.java
│       └── stepdefs/
│           └── CustomerStepDefinitions.java
└── resources/
    └── features/
        └── customer_management.feature
```

This organization clearly separates .feature files from Java configuration classes and step definitions.

5.8.2 Spring Configuration Class for Cucumber

Create the file:

digibank-web/src/test/java/com/m2ibank/web/cucumber/CucumberSpringConfiguration.java

```
package com.m2ibank.web.cucumber;
```

```
import org.springframework.boot.test.context. SpringBootTest ;

import org.springframework.test.context. ActiveProfiles ;

import io.cucumber.spring. CucumberContextConfiguration ;


@CucumberContextConfiguration

@SpringBootTest ( webEnvironment =
SpringBootTest.WebEnvironment.RANDOM_PORT )

@ActiveProfiles ( "test" )

public class CucumberSpringConfiguration {

}
```

5.8.3 Cucumber Launch Class

Create the file:

digibank-web/src/test/java/com/m2ibank/web/cucumber/RunCucumberTest.java

```
package com.m2ibank.web.cucumber;


import org.junit.platform.suite.api. ConfigurationParameter ;

import org.junit.platform.suite.api. IncludeEngines ;

import org.junit.platform.suite.api. SelectClasspathResource ;

import org.junit.platform.suite.api. Following ;


import static io.cucumber.junit.platform.engine.Constants.
GLUE_PROPERTY_NAME ;
```

```
import static io.cucumber.junit.platform.engine.Constants.
PLUGIN_PROPERTY_NAME ;


@Following

@IncludeEngines ( "cucumber" )

@SelectClasspathResource ( "features" )

@ConfigurationParameter (key = GLUE_PROPERTY_NAME , value =
"com.m2ibank.web.cucumber" )

@ConfigurationParameter (key = PLUGIN_PROPERTY_NAME , value =
"pretty, summary" )

public class RunCucumberTest {

}
```

5.8.4 First Cucumber Scenario

Create the file:

digibank-web/src/test/resources/features/customer_management.feature

```
Feature : Customer management


 Scenario : Create a new customer successfully

   Given the customer payload is valid

   When the client submits the customer creation request

   Then the response status should be 201

   And the response should indicate a successful customer
creation
```

5.8.5 Defining the steps

Create the file:

digibank-web/src/test/java/com/m2ibank/web/cucumber/stepdefs/CustomerStepDefinitions.java

```
package com.m2ibank.web.cucumber.stepdefs;


import com.fasterxml.jackson.databind.ObjectMapper;

import com.m2ibank.customer.dto.CustomerRequest;

import io.cucumber.java.en. And ;

import io.cucumber.java.en. Given ;

import io.cucumber.java.en. Then ;

import io.cucumber.java.en. When ;

import org.springframework.beans.factory.annotation. Autowired ;

import
org.springframework.boot.test.web.client.TestRestTemplate;

import org.springframework.boot.test.web.server. LocalServerPort
;

import org.springframework.http.*;


import static org.junit.jupiter.api.Assertions.*;


public class CustomerStepDefinitions {


   @LocalServerPort
```

```
   private int port ;


   @Autowired
   private TestRestTemplate restTemplate ;


   @Autowired
   private ObjectMapper objectMapper ;


   private CustomerRequest request ;
   private ResponseEntity<String> response ;


   @Given ( "the customer payload is valid" )
   public void theCustomerPayloadIsValid () {
       request = new CustomerRequest();
       request .setFullName( "Cucumber User" );
       request .setEmail( "cucumber@m2ibank.com" );
       request .setPhoneNumber( "+237622222222" );
       request .setNationalId( "CNI-CUC-001" );
}


   @When ( "the client submits the customer creation request" )
   public void theClientSubmitsTheCustomerCreationRequest ()
throws Exception {
HttpHeaders headers = new HttpHeaders();
```

```
headers.setContentType(MediaType.APPLICATION_JSON ) ;


HttpEntity<String> entity = new HttpEntity<>(

                objectMapper.writeValueAsString ( request ),

headers

);


        response = restTemplate.postForEntity (

                "http://localhost:" + port + "/api/customers" ,

entity,

String.class

        );

}


   @Then ( "the response status should be 201" )

   public void theResponseStatusShouldBe201 () {

        assertEquals (HttpStatus.CREATED , response.getStatusCode

( ));

}


   @And ( "the response should indicate a successful customer

creation" )

   public void

theResponseShouldIndicateASuccessfulCustomerCreation () {

        assertNotNull ( response.getBody ());
```

```
        assertTrue ( response .getBody().contains( " \" success
\" :true" ));

        assertTrue ( response .getBody().contains( "Customer
created successfully" ));

}

}
```

This first scenario demonstrates end-to-end client creation. It already gives students a clear idea of the value of Cucumber: describing the expected behavior, then linking it to a real-world execution.

5.9 Second recommended Cucumber scenario: invalid transfer

To take this a step further, it is strongly recommended to add a second business scenario demonstrating that a transfer cannot be made to the same account. This clearly illustrates the concept of a business rule expressed in functional language.

Create the file:

digibank-web/src/test/resources/features/transfer_management.feature

```
Feature : Transfer management


 Scenario : Reject transfer when source and destination accounts
are identical

   Given a transfer payload with the same source and destination
account

   When the client submits the transfer request

   Then the transfer response status should be 400

   And the transfer response should contain the business error
message
```

Create the file:

digibank-web/src/test/java/com/m2ibank/web/cucumber/stepdefs/TransferStepDefinitions.java

```
package com.m2ibank.web.cucumber.stepdefs;


import com.fasterxml.jackson.databind.ObjectMapper;

import com.m2ibank.transfer.dto.TransferRequest;

import io.cucumber.java.en. And ;

import io.cucumber.java.en. Given ;

import io.cucumber.java.en. Then ;

import io.cucumber.java.en. When ;

import org.springframework.beans.factory.annotation. Autowired ;

import
org.springframework.boot.test.web.client.TestRestTemplate;

import org.springframework.boot.test.web.server. LocalServerPort
;

import org.springframework.http.*;


import java.math.BigDecimal;


import static org.junit.jupiter.api.Assertions.*;


public class TransferStepDefinitions {


   @LocalServerPort
```

```
    private int port ;


    @Autowired
    private TestRestTemplate restTemplate ;


    @Autowired
    private ObjectMapper objectMapper ;


    private TransferRequest request ;
    private ResponseEntity<String> response ;


    @Given ( "a transfer payload with the same source and
destination account" )
    public void
a_transfer_payload_with_the_same_source_and_destination_account
() {
        request = new TransferRequest();
        request .setSourceAccountId( 100L );
        request .setDestinationAccountId( 100L );
        request .setAmount ( BigDecimal.valueOf ( 500 ));
        request .setDescription( "Test identical accounts" );
}


    @When ( "the client submits the transfer request" )
```

```
   public void the_client_submits_the_transfer_request () throws
Exception {

HttpHeaders headers = new HttpHeaders();

headers.setContentType(MediaType.APPLICATION_JSON ) ;


HttpEntity<String> entity = new HttpEntity<>(

               objectMapper.writeValueAsString ( request ),

headers

);


       response = restTemplate.postForEntity (

               "http://localhost:" + port + "/api/transfers" ,

entity,

String.class

       );

}


   @Then ( "the transfer response status should be {int}" )

   public void the_transfer_response_status_should_be (Integer
statusCode) {

       assertEquals (statusCode, response.getStatusCode
().value());

}
```

```
   @And ( "the transfer response should contain the business
error message" )

   public void
the_transfer_response_should_contain_the_business_error_message
() {

       assertNotNull ( response.getBody ());

       assertTrue ( response .getBody().contains( "Source and
destination accounts must be different" ));

}

}
```

5.10 Maven validation commands

Once the tests are written, students must be taught how to run them progressively. The following commands should be included in the workshop document.

5.10.1 Run all tests

```
mvn test
```

5.10.2 Launch the full build

```
mvn clean install
```

5.10.3 Run only the tests of a module

```
mvn -pl customer-module test

mvn -pl account-module test
```

```
mvn -pl transfer-module test

mvn -pl digibank-web test
```

5.10.4 Launch the application after validation

```
mvn spring-boot:run -pl digibank-web
```

These commands allow students to clearly distinguish between local control of a module and validation of the entire project. In a real DevSecOps context, this discipline is essential.

5.11 Preparation of evidence of execution

In a graded or demonstrated workshop, simply having tests in the project is not always enough. It is also necessary to be able to prove that they execute correctly and that they actually validate the expected behavior.

Evidence of completion that could be requested from students might include the following:

- Terminal capture after mvn test
- Cucumber execution capture
- Swagger UI screenshot
- Capturing a successful customer creation
- Capture of a valid transfer
- Screenshot of a refused transfer or failed validation
- Optional export of the Maven or IDE test report.

This requirement has real educational value. It pushes students not just to "write tests", but to run them, interpret them and use them as proof of software quality.

5.11 Expected results at this stage

At the end of this section, DigiBank should have:

- Unit tests on core services
- At least one Spring Boot integration test
- At least one executable Cucumber scenario
- Clearly identified Maven commands to initiate validations
- From a simple proof-of-execution protocol.

This level of coverage is intentionally introductory, but it is already sufficient to show students that a serious modular banking project is not limited to "making endpoints work." It must also demonstrate, through testing, that its business rules are controlled and that its behavior is stable.

Part 6: Dockerfiles, Docker Compose, GitHub Actions pipeline, project packaging, and final validation of the complete execution

6.1 Role of this phase

At this stage of the workshop, DigiBank is already able to start, expose its REST APIs, present an index view, document its endpoints via Swagger, and run unit tests, initial integration tests, and Cucumber scenarios. However, one crucial step remains: transforming this project into a truly industrializable artifact—that is, one that is buildable, portable, containerizable, and automatically verifiable.

This sixth part addresses this objective. It introduces packaging mechanisms, application containerization, local orchestration with Docker Compose, and build automation via GitHub Actions. It also demonstrates to students that a modern application is not limited to simply "working in the IDE," but must be able to run in a standardized environment and be validated by an automated pipeline.

Even though this phase is intentionally kept simple compared to a full industrial environment, it already provides an excellent introduction to the fundamentals of modern deployment. It directly prepares participants for subsequent workshops on container security, dependency security, and pipeline hardening.

6.2 Packaging Maven final project

The first step is to ensure that DigiBank can be properly packaged using Maven. In the chosen architecture, the digibank-web module represents the executable entry point, while the other modules are packaged as internal libraries within the modular monolith.

6.2.1 Verification of the main module packaging

In digibank-web/pom.xml, the packaging should remain in JAR format for simple execution with Spring Boot. This choice is consistent with the desire to facilitate local execution, Docker, and the automation of the workflow.

The following section must be present:

```
<packaging> jar </packaging>
```

The Spring Boot plugin must also be correctly declared:

```
<build>
<plugins>
<plugin>
<groupId> org.springframework.boot </groupId>
<artifactId> spring-boot-maven-plugin </artifactId>
<configuration>
<mainClass> com.m2ibank.web.DigiBankApplication </mainClass>
</configuration>
<executions>
<execution>
<goals>
<goal> repackage </goal>
</goals>
</execution>
</executions>
</plugin>
</plugins>
</build>
```

This configuration allows Maven to produce an executable artifact incorporating the necessary dependencies.

6.2.2 Global Packaging Order

From the project root, the student must be able to execute:

```
mvn clean package
```

Or, if you also want to run all the tests and install the artifacts locally:

```
mvn clean install
```

After execution, the main executable file should be available in:

```
digibank-web/target/digibank-web-1.0.0-SNAPSHOT.jar
```

This step is important because it marks the transition between “development project” and “deliverable artifact”.

6.3 Running the JAR locally

Before containerizing the application, it is necessary to verify that it works in standalone mode from the generated JAR.

The expected order is:

```
Java -jar digibank-web/target/digibank-web-1.0.0-SNAPSHOT.jar
```

If PostgreSQL is already running and accessible, the application should start correctly, initialize its Spring context, connect to the database, and make the following available:

- the index view;
- Swagger UI;
- REST endpoints;
- the initialization data.

This verification is essential, because Docker must not become a cover-up masking a faulty application packaging.

6.4 Creating the application's Dockerfile

DigiBank containerization needs to be introduced properly. The goal is not yet to build an ultra-optimized and hardened image like in the container security workshop, but to offer a first functional, clear and pedagogically readable image.

Create the following file in the root directory of the project:

Dockerfile

```
FROM eclipse-temurin:17-jdk-alpine

WORKDIR / app

COPY digibank-web / target / digibank-web-1.0.0-SNAPSHOT.jar
app.jar

EXPOSE 8080

ENTRYPOINT [ "java" , "-jar" , "app.jar" ]
```

This Dockerfile is intentionally simple. It relies on a lightweight Java 17 image, sets a clear working directory, copies the artifact generated by Maven, and then launches the application.

However, it is important to explain to students that this version is just a first step. Later, in a security and optimization workshop, the aim will be to:

- Use a multi-stage build
- Reduce the image size further
- Avoid running the container with unnecessary privileges.
- Outsource configuration and secrets more cleanly.

6.5 Building the Docker image

Once the JAR is generated and the Dockerfile is created, the image can be built.

From the project's origin:

```
mvn clean package

docker build -t digibank-app:1.0.0 .
```

After construction, the student can check for the presence of the image:

```
docker images
```

This command should show an image named digibank-app with the tag 1.0.0.

6.6 Manually launching the application container

If PostgreSQL is already running on the host machine or in another accessible container, the application can be launched directly with Docker.

```
docker run --name digibank-app \

 -p 8080:8080 \
```

```
                                                                    -e
SPRING_DATASOURCE_URL=jdbc:postgresql://host.docker.internal:543
2/digibankdb \
 -e SPRING_DATASOURCE_USERNAME=digibank \
 -e SPRING_DATASOURCE_PASSWORD=digibank123 \
 digibank-app:1.0.0
```

On some Linux environments, host.docker.internal may require adjustments, which justifies the use of Docker Compose to simplify orchestration.

6.7 Creating docker-compose.yml

To make execution more reproducible and closer to a standardized environment, it is best to define the application and the database in a Docker Compose file.

Create the file:

docker-compose.yml

```
version : "3.9"


services :
 postgres :
   image : postgres:16
   container_name : digibank-postgres
   restart : unless stopped
   environment :
     POSTGRES_DB : digibankdb
     POSTGRES_USER : digibank
```

```
    POSTGRES_PASSWORD : digibank123
  ports :
- "5432:5432"
  volumes :
- digibank_postgres_data:/var/lib/postgresql/data


 digibank-app :
  build :
    context : .
    Dockerfile : Dockerfile
  container_name : digibank-app
  depends_on :
- postgres
  restart : unless stopped
  environment :
                                    SPRING_DATASOURCE_URL       :
jdbc:postgresql://postgres:5432/digibankdb
    SPRING_DATASOURCE_USERNAME : digibank
    SPRING_DATASOURCE_PASSWORD : digibank123
    SPRING_PROFILES_ACTIVE : dev
  ports :
- "8080:8080"


volumes :
```

```
digibank_postgres_data :
```

This file allows you to launch the entire system locally with a single command. It plays a very important educational role, as it demonstrates how to transform several components into a coherent executable environment.

6.8 Essential Docker Compose Commands

Students must learn to manipulate the following basic commands:

6.8.1 Construction and Start-up

```
docker compound up --build
```

6.8.2 Background startup

```
docker compound up -d --build
```

6.8.3 Consulting the containers

```
docker compound ps
```

6.8.4 Consulting newspapers

```
docker compound logs -f
```

6.8.5 Environmental shutdown

```
docker compound down
```

6.8.6 Shutdown with volume deletion

```
docker compound down -v
```

These commands are important because they form the operational basis of any local containerized execution.

6.9 Functional verification after containerization

Once docker compose up --build has been successfully executed, the student should verify:

- The PostgreSQL container has started
- The DigiBank container has started
- The application should respond correctly on port 8080.
- The initialization data has been injected.
- That Swagger UI remains accessible
- That the business endpoints still function as expected.

The URLs to test are as follows:

```
http://localhost:8080/

http://localhost:8080/swagger-ui.html

http://localhost:8080/api/customers
```

This step confirms that the application is not only executable in the IDE, but also in a reproducible containerized environment.

6.10 Creating the GitHub Actions Pipeline

Once local containerization is validated, at least the basic quality assurance operations must be automated. In this workshop, the GitHub Actions pipeline must, at a minimum:

- Retrieve the source code

- Install Java
- Run Maven
- Launch the tests
- Build the project.

Create the file:

.github/workflows/digibank-ci.yml

```
Name : DigiBank CI Pipeline


on :
 push :
   branches :
- hand
- master
- develop
 pull_request :
   branches :
- hand
- master
- develop


jobs :
 build-and-test :
   runs-on : ubuntu-latest


   steps :
```

```
- name : Checkout source code
       uses : actions/checkout@v4


- name : Set up Java 17
       uses : actions/setup-java@v4
       with :
         distribution : temurin
         Java version : 17
         cache : maven


- name : Build and run tests
       run : mvn clean verify
```

This first pipeline is intentionally simple, but it is already very useful for teaching purposes. It shows that the application must be able to be automatically rebuilt and validated outside of the developer's local machine.

6.11 Recommended Pipeline Enrichment

A more complete version can also build the Docker image after testing. This isn't essential in a first version, but it's a good improvement to highlight in the documentation.

Here is an enhanced version:

```
Name : DigiBank CI Pipeline


on :
 push :
```

```
   branches :
- hand
- master
- develop
 pull_request :
   branches :
- hand
- master
- develop

jobs :
 build-test-package :
   runs-on : ubuntu-latest

   steps :
- name : Checkout source code
       uses : actions/checkout@v4

- name : Set up Java 17
       uses : actions/setup-java@v4
       with :
         distribution : temurin
         Java version : 17
         cache : maven
```

```
- name : Build and run tests
      run : mvn clean verify

- name : Build JAR package
      run : mvn clean package -DskipTests

- name : Build Docker image
      run : docker build -t digibank-app:ci .
```

This version explicitly adds the packaging and image building steps. It strengthens the consistency between application builds and containerized builds.

6.12 Files to include in the final submission

For a DigiBank project to be considered complete at the end of workshop 1, the Git repository must contain at least:

- The parent and module pom.xml files
- Java source code
- Configuration resources
- Thymeleaf templates
- JUnit tests
- Integration tests
- The Cucumber scenarios
- The Dockerfile
- The docker-compose.yml file
- The GitHub Actions pipeline
- A README.md explaining how to run the project.

This clearly shows that the workshop is not only about coding, but about delivering a complete, organized and re-executable project.

6.13 Minimum recommended content of README.md

Although the detailed writing of the README can be done separately, the workshop document must indicate what it should contain.

The minimum recommended README should include:

- The name of the project
- His goal
- The multi-module architecture
- Software prerequisites
- The main Maven commands
- Docker and Docker Compose commands
- The URL of the homepage
- The Swagger UI URL
- The main APIs available
- Test commands.

This requirement prepares students to actually document their deliverables, and not just produce source code.

6.14 Final and complete validation of the project

At the end of the workshop, the student must be able to demonstrate a complete and consistent execution chain. The final validation must follow a simple, explicit, and reproducible sequence.

6.14.1 Build validation

```
mvn clean verify
```

This command must compile, test and validate the entire project **.**

6.14.2 Packaging validation

```
mvn clean package
```

This command should produce the JAR file for the main module.

6.14.3 Validation of local execution

```
mvn spring-boot:run -pl digibank-web
```

Then check the endpoints and Swagger.

6.14.4 Validation of containerized execution

```
docker compound up --build
```

Then test the same functionalities from the browser or Swagger UI.

6.14.5 CI Pipeline Validation

Once the repository is pushed to GitHub, the student must verify that the workflow runs correctly and that the steps pass without error.

Part 7: Generating an initial dataset with Flyway

7.1 Why add Flyway to DigiBank

In the early stages of an educational project, the boot data is often created by Java code launched at startup, for example via a CommandLineRunner or an ApplicationRunner. This approach is simple at first, but it quickly becomes difficult to maintain as soon as you want to version the schema, cleanly replay environments, guarantee consistency across multiple workstations, or run the application in Docker and in a CI/CD pipeline.

Flyway addresses this need precisely. It allows you to manage database schema evolution and data initialization using ordered and versioned SQL scripts. In DigiBank, the introduction of Flyway ensures that every learner starts with the same table structure and demo dataset, facilitating JUnit testing, Cucumber scenarios, Swagger validations, Docker Compose trials, and proof-of-execute preparation.

The addition of Flyway is therefore useful from both a technical and pedagogical point of view. Technically, it ensures the reproducibility of the environment. Pedagogically, it introduces students to a practice that is now standard in modern Java projects based on Spring Boot and PostgreSQL.

7.2 Objective of this section

The goal of this final section is to replace or supplement the manual data initialization mechanisms with a versioned solution using Flyway. At the end of this step, DigiBank should be able to:

- Automatically create its relational schema at startup;
- Inject a consistent dataset for customers, accounts and transfers;
- Replay this data identically on multiple machines;
- Working cleanly with PostgreSQL, Docker Compose and the GitHub Actions pipeline.

This step logically concludes the workshop, as it provides a stable environment for all tests and for all future executions of the project.

7.3 Maven Flyway Dependency

The first step is to add Flyway to the main module that carries the application startup, usually digibank-web.

In digibank-web/pom.xml, add the following dependency:

```
<dependency>
<groupId> org.flywaydb </groupId>
<artifactId> flyway-core </artifactId>
</dependency>
```

If PostgreSQL is used, as is the case here, it is also recommended to add:

```
<dependency>
<groupId> org.flywaydb </groupId>
<artifactId> flyway-database-postgresql </artifactId>
</dependency>
```

Once the dependencies are added, run:

```
mvn clean install
```

This command will verify that the project compiles correctly with Flyway integrated.

7.4 Spring Boot configuration for Flyway

Next, Flyway needs to be activated and configured in the application's configuration files.

In digibank-web/src/main/resources/application-dev.yml, add or complete the following properties:

```
spring :
 datasource :
   url : jdbc:postgresql://localhost:5432/digibankdb
   username : digibank
   password : digibank123
   driver-class-name : org.postgresql.Driver


 jpa :
   hibernate :
     ddl-auto : validate
   show-sql : true
   properties :
     hibernate :
       SQL format : true


 flyway :
   enabled : true
   baseline-on-migrate : true
   locations : classpath:db/migration
```

This configuration has several important implications:

- **flyway.enabled: true** enables migrations
- **baseline-on-migrate: true** facilitates initialization on certain existing databases

- **locations** indicates where Spring Boot should look for Flyway scripts
- **ddl-auto:valid** asks Hibernate to check the consistency of the schema without trying to recreate it itself.

This last point is important: if Flyway manages the schema, it's best to prevent Hibernate from attempting to generate it automatically in parallel. This shifts us from an implicit generation logic to a versioned and controlled one.

7.5 Flyway Script Tree

Create the following directory in the digibank-web module:

```
src/main/resources/db/migration
```

All Flyway scripts must be placed in this folder, with a strict naming convention.

The recommended convention is:

- **V1__create_schema.sql**
- **V2__insert_seed_data.sql**

The prefix V stands for “version”, followed by a version number, then a double underscore and a descriptive name. This convention is important because Flyway relies on it to order the execution of scripts.

7.6 Creating the schema script V1__create_schema.sql

The first script must create the tables necessary for the minimum operation of DigiBank.

Create the file:

digibank-web/src/main/resources/db/migration/V1__create_schema.sql

```
CREATE TABLE IF NOT EXISTS customers (

id BIGSERIAL PRIMARY KEY ,
```

```
full_name VARCHAR ( 150 ) NOT NULL ,
email VARCHAR ( 150 ) NOT NULL UNIQUE ,
phone_number VARCHAR ( 30 ) NOT NULL UNIQUE ,
national_id VARCHAR ( 100 ) NOT NULL UNIQUE ,
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
   );

CREATE TABLE IF NOT EXISTS accounts (
id BIGSERIAL PRIMARY KEY ,
account_number VARCHAR ( 50 ) NOT NULL UNIQUE ,
balance NUMERIC ( 19 , 2 ) NOT NULL ,
account_type VARCHAR ( 30 ) NOT NULL ,
customer_id BIGINT NOT NULL ,
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ,
   CONSTRAINT fk_accounts_customer
   FOREIGN KEY (customer_id)
   REFERENCES customers(id)
   ON DELETE CASCADE
   );

CREATE TABLE IF NOT EXISTS transfers (
id BIGSERIAL PRIMARY KEY ,
source_account_id BIGINT NOT NULL ,
destination_account_id BIGINT NOT NULL ,
```

```
amount NUMERIC ( 19 , 2 ) NOT NULL ,

description VARCHAR ( 255 ),

transfer_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ,

status VARCHAR ( 30 ) NOT NULL ,

   CONSTRAINT fk_transfer_source_account

   FOREIGN KEY (source_account_id)

   REFERENCES accounts(id)

   ON DELETE CASCADE ,

   CONSTRAINT fk_transfer_destination_account

   FOREIGN KEY (destination_account_id)

   REFERENCES accounts(id)

   ON DELETE CASCADE

   );
```

This script creates a minimal schema consistent with the entities manipulated in the workbench. It already supports:

- The customers
- The accounts
- Bank transfers.

The student must understand that in a larger project, each schema change should result in a new versioned script instead of editing the old ones. This allows for a clear history of database evolution.

7.7 Creating the data script V2__insert_seed_data.sql

The second script will inject a starting dataset that can be used in Swagger, in integration tests and in end-of-workshop demonstrations.

Create the file:

digibank-web/src/main/resources/db/migration/V2__insert_seed_data.sql

```
INSERT INTO customers ( id , full_name , email , phone_number ,
national_id , created_at )

VALUES

    ( 1 , 'Alice Ndzi' , 'alice@m2ibank.com' , '+237600000001' ,
'CNI000001' , CURRENT_TIMESTAMP ),

( 2 , 'Brice Tchoumi' , 'brice@m2ibank.com' , '+237600000002' ,
'CNI000002' , CURRENT_TIMESTAMP ),

(   3   ,   'Clarisse   Mvondo'   ,   'clarisse@m2ibank.com'   ,
'+237600000003' , 'CNI000003' , CURRENT_TIMESTAMP )

   ON CONFLICT (id) DO NOTHING;


INSERT  INTO  accounts  (  id  ,  account_number  ,  balance  ,
account_type , customer_id , created_at )

VALUES

      (  1  ,  'DB-ACC-0001'  ,  150000.00  ,  'CURRENT'  ,  1  ,
CURRENT_TIMESTAMP ),

(   2   ,   'DB-ACC-0002'   ,   250000.00   ,   'SAVINGS'   ,   2   ,
CURRENT_TIMESTAMP ),

(   3   ,   'DB-ACC-0003'   ,   50000.00   ,   'CURRENT'   ,   3   ,
CURRENT_TIMESTAMP )

   ON CONFLICT (id) DO NOTHING;


INSERT   INTO   transfers   (   id   ,   source_account_id   ,
destination_account_id , amount , description , transfer_date ,
status )

VALUES
```

```
    ( 1 , 1 , 2 , 10000.00 , 'Initial transfer for testing' ,
CURRENT_TIMESTAMP , 'SUCCESS' ),

( 2 , 2 , 3 , 15000.00 , 'Savings to current transfer' ,
CURRENT_TIMESTAMP , 'SUCCESS' )

   ON CONFLICT (id) DO NOTHING;
```

This dataset offers several advantages:

- He immediately supplied several customers
- He creates several accounts of different types
- It already offers a transfer history
- It provides a concrete basis for testing endpoints without having to re-enter everything manually.

Choosing ON CONFLICT DO NOTHING helps avoid certain reinsertion problems if the database already contains the same keys, even though in practice Flyway doesn't replay a script that has already been migrated. This remains an acceptable pedagogical precaution in a practical exercise context.

7.8 Adjusting PostgreSQL Sequences

When explicit identifiers are manually inserted into BIGSERIAL columns, it is prudent to realign the PostgreSQL sequences so that future automatic inserts resume correctly.

At the end of the V2__insert_seed_data.sql file, add:

```
SELECT setval ( 'customers_id_seq' , ( SELECT MAX ( id ) FROM
customers));

SELECT setval ( 'accounts_id_seq' , ( SELECT MAX ( id ) FROM
accounts));

SELECT setval ( 'transfers_id_seq' , ( SELECT MAX ( id ) FROM
transfers));
```

This step prevents a future insert without an explicit identifier from attempting to reuse a value that is already present.

7.9 Deletion or modification of the CommandLineRunner

If the application previously used a CommandLineRunner to create data at startup, it now needs to be removed or simplified.

Two approaches are possible:

- Completely remove this mechanism and let Flyway manage the dataset on its own.
- Retain a Java initialization component, but only for very specific demonstrations not covered by SQL.

Within the workshop, the first approach is the cleanest. It simplifies startup behavior and avoids duplication between Java initialization and SQL initialization.

7.10 Checking the start-up with Flyway

Once the scripts are created, the student must restart the application on a clean basis.

The recommended sequence is as follows:

7.10.1 Recreate the database or clear the environment

If PostgreSQL is running in Docker Compose:

```
docker compound down -v

docker compound up -d postgres
```

Or, if PostgreSQL is local, delete and then recreate the digibankdb database.

7.10.2 Restart the application

```
mvn spring-boot:run -pl digibank-web
```

At startup, Flyway must:

- Detect the absence of the pattern
- Run V1__create_schema.sql
- Run V2__insert_seed_data.sql
- Record the migrations in its history table.

In the logs, the student should observe messages indicating the successful execution of the migrations.

7.11 Functional verification of injected data

After starting, students must verify that the data is accessible through the APIs already created.

The following calls should produce consistent responses:

```
GET http://localhost:8080/api/customers

GET http://localhost:8080/api/accounts

GET http://localhost:8080/api/transfers
```

The results should show:

- 3 clients
- 3 accounts
- 2 existing transfers.

This verification is important because it proves that the schema and data are truly aligned with the business code and with the exposed endpoints.

7.12 Verification with Docker Compose

Adding Flyway becomes particularly useful in a containerized environment. Once docker-compose.yml is in place, the student can run:

```
docker compound down -v

docker compound up --build
```

In this case, PostgreSQL starts in a clean state, and then DigiBank automatically applies its Flyway migrations at startup. The application is therefore immediately usable without any manual intervention on the database.

This is an excellent demonstration point in the workshop, as it concretely shows the value of automation and reproducible infrastructure.

7.13 Interest in testing and execution proofs

Using Flyway also enhances the reliability of tests and demonstrations. When a student runs integration tests or Cucumber scenarios, they know the database has been initialized to a known state. Similarly, when preparing Swagger UI captures or proof-of-transfers, they no longer rely on a manually prepared environment.

In other words, Flyway is not just used to "load some data". It helps stabilize the educational execution context, making the entire practical exercise more robust, more comparable from one workstation to another, and easier to correct.

7.14 Update of requested proofs of execution

With Flyway, it becomes relevant to add new proofs of execution to deliverables or presentations:

- Capture of logs showing the execution of Flyway migrations
- Capture of the flyway_schema_history history table
- Capture of the /api/customers API showing the injected data
- Capture of the /api/accounts API displaying the demo accounts

- Capture of the /api/transfers API showing the initial transfer history.

These proofs usefully complement those already planned for the build, Docker Compose, Swagger and GitHub Actions.

7.15 Expected result after adding Flyway

At the end of this last part, DigiBank should be able to start on a clean PostgreSQL database, automatically create its relational schema, inject a consistent initial dataset and make this dataset immediately accessible via the API and Swagger.

The addition of Flyway neatly concludes Workshop 1, as it transforms the project into a complete educational base: structured, tested, documented, packaged, containerized, automated and now reproducible at the database level.

Part 8: Expected Outcome of Workshop 1

8.1 Final evidence of performance to be requested

To create a complete deliverable or presentation support document, it is relevant to request the following evidence:

- Capture of mvn clean verify successful
- Capture of docker compose up --build successful
- Screenshot of the homepage
- Swagger UI screenshot
- Screenshot of a customer creation test
- Screenshot of an account creation test
- Capture of a successful transfer
- Capture of GitHub Actions execution
- Final project tree in IntelliJ IDEA.

These elements demonstrate that the workshop was carried out to completion and that the project actually works in several execution contexts.

8.2 Expected result at the end of workshop 1

By the end of this sixth part, DigiBank should be a project:

- Structured into business modules
- Executable locally
- Documented via Swagger
- Partially tested with JUnit, integration, and Cucumber
- Packaged in JAR format
- Containerized with Docker
- Orchestrable with Docker Compose

- Automatically validated with GitHub Actions.

This result is perfectly consistent with the objective of workshop 1: to build a first solid, usable and clean base, which will serve as a support for the following workshops devoted to static analysis, dynamic analysis and the security of containers and dependencies.